

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ
ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Τομέας Επικοινωνιών, Ηλεκτρονικής και Συστημάτων Πληροφορικής
Εργαστήριο Διαχείρισης και Βέλτιστου Σχεδιασμού Δικτύων Τηλεματικής - NETMODE

Συστήματα Αναμονής
5^η Εργαστηριακή Άσκηση



Στοιχεία Φοιτητή

- Όνομα: Πέππας Μιχαήλ-Αθανάσιος
- Αριθμός Μητρώου: 03121026
- 4^η Εργαστηριακή Ομάδα (Δευτέρα, 12:45-14:30)
- Ημερομηνία Διεξαγωγής Εργαστηρίου: 20.04.2026
- Ημερομηνία Παράδοσης Αναφοράς: 26.04.2026

Περιεχόμενα

1	Προσομοίωση διακριτών γεγονότων Web Server	3
1.1	Περιγραφή και μοντελοποίηση του συστήματος	3
1.2	Δομή της προσομοίωσης διακριτών γεγονότων	4
1.3	Ερώτημα A.1: Εκτέλεση για $\lambda = 100$ requests/sec	6
1.4	Ερώτημα A.2: Στασιμότητα για διαφορετικές τιμές του λ	9
1.5	Ερώτημα A.3: Διαστήματα εμπιστοσύνης και αφαίρεση μεταβατικής περιόδου	11
1.5.1	Μέθοδος αφαίρεσης transient	11
1.5.2	Αποτελέσματα διαστημάτων εμπιστοσύνης	13
1.6	Ερώτημα A.4: Επαλήθευση του νόμου Little	15
1.7	Συνολικό συμπέρασμα πρώτου μέρους	16
2	Ανοιχτό δίκτυο ουρών αναμονής	18
2.1	Περιγραφή του δικτύου και παραδοχές Jackson	18
2.2	Μεταβάσεις και πιθανότητες δρομολόγησης	19
2.3	Ερώτημα B.2: Εξισώσεις ροής και εντάσεις φορτίου	19
2.4	Ερώτημα B.3: Μέσος αριθμός πελατών σε κάθε ουρά	21
2.5	Ερώτημα B.4: Αριθμητική εφαρμογή για τις δοθείσες παραμέτρους	21
2.6	Ερώτημα B.5: Στενωπός και μέγιστη τιμή του λ	24
2.7	Ερώτημα B.6: Διάγραμμα καθυστέρησης ως προς λ	26
2.8	Συνολικό συμπέρασμα δεύτερου μέρους	28
3	Παράρτημα Υλοποίησης σε Octave	30
3.1	Κώδικας Υλοποίησης 1ου μέρους	30
3.2	Έξοδος Προγράμματος 1ου μέρους	46
3.3	Κώδικας Υλοποίησης 2ου μέρους	50
3.4	Έξοδος Προγράμματος 2ου μέρους	59

1 Προσομοίωση διακριτών γεγονότων Web Server

1.1 Περιγραφή και μοντελοποίηση του συστήματος

Στο πρώτο μέρος της άσκησης μελετάμε μέσω προσομοίωσης διακριτών γεγονότων έναν Web Server, ο οποίος εξυπηρετεί αιτήματα με σειρά FIFO. Κάθε request που φτάνει στο σύστημα δημιουργεί αρχικά μία εργασία τύπου 1. Όταν ολοκληρωθεί η εργασία τύπου 1, το ίδιο request δεν αποχωρεί από το σύστημα, αλλά δημιουργεί μία εργασία τύπου 2, η οποία τοποθετείται ξανά στο τέλος της ίδιας ουράς. Το request θεωρείται πλήρως εξυπηρετημένο μόνο όταν ολοκληρωθεί και η εργασία τύπου 2.

Η βασική ιδιαιτερότητα του μοντέλου είναι ότι οι δύο φάσεις του ίδιου request δεν εξυπηρετούνται απαραίτητα διαδοχικά. Η δεύτερη φάση εισέρχεται ξανά στο τέλος της FIFO ουράς και μπορεί να προηγούνται άλλες εργασίες, είτε τύπου 1 είτε τύπου 2. Επομένως, το σύστημα είναι ένας single-server FIFO μηχανισμός με δύο φάσεις εξυπηρέτησης ανά request.

Οι αφίξεις ακολουθούν διαδικασία Poisson με ρυθμό λ requests/sec. Συνεπώς, οι χρόνοι μεταξύ διαδοχικών αφίξεων είναι εκθετικά κατανομημένοι. Επειδή η προσομοίωση υλοποιείται σε milliseconds, ο μέσος χρόνος μεταξύ αφίξεων είναι:

$$E[A] = \frac{1000}{\lambda} \text{ ms.}$$

Οι χρόνοι εξυπηρέτησης των δύο τύπων εργασιών είναι ανεξάρτητοι και ακολουθούν διαφορετικές κατανομές:

$$S_1 \sim \text{LogNormal}(\mu = 1, \sigma = 1),$$

και:

$$S_2 \sim \text{Uniform}(0.5, 1).$$

Για την εργασία τύπου 1, επειδή η λογαριθμοκανονική κατανομή προκύπτει ως $X = e^Y$, με $Y \sim N(\mu, \sigma^2)$, η μέση τιμή της είναι:

$$E[S_1] = e^{\mu + \sigma^2/2} = e^{1+1/2} = e^{1.5} \simeq 4.481689 \text{ ms.}$$

Για την εργασία τύπου 2 έχουμε:

$$E[S_2] = \frac{0.5 + 1}{2} = 0.75 \text{ ms.}$$

Άρα, το μέσο συνολικό service requirement ανά request είναι:

$$E[S] = E[S_1] + E[S_2] \simeq 5.231689 \text{ ms.}$$

Η προσομοίωση είναι μη τερματική ως προς τη φύση του συστήματος, αλλά κάθε run σταματά τεχνητά όταν εξυπηρετηθούν πλήρως 10^4 requests. Η βασική κατάσταση που διατηρεί ο προσομοιωτής περιλαμβάνει τον τρέχοντα χρόνο, την κατάσταση του server, τη FIFO ουρά, τους μετρητές αφίξεων και ολοκληρωμένων requests, καθώς και τα χρονικά logs που απαιτούνται για τον υπολογισμό των μέσων μεγεθών.

1.2 Δομή της προσομοίωσης διακριτών γεγονότων

Στην προσομοίωση διακριτών γεγονότων ο χρόνος δεν προχωρά με σταθερό βήμα, αλλά μετακινείται από γεγονός σε γεγονός. Στο συγκεκριμένο πρόβλημα υπάρχουν τρεις τύποι γεγονότων:

- **ARRIVAL**: φτάνει νέο request και δημιουργείται εργασία τύπου 1.
- **DEPART1**: ολοκληρώνεται μία εργασία τύπου 1 και το ίδιο request δημιουργεί εργασία τύπου 2.
- **DEPART2**: ολοκληρώνεται μία εργασία τύπου 2 και το request αποχωρεί από το σύστημα.

Στο γεγονός ARRIVAL αυξάνεται ο μετρητής αφίξεων και προγραμματίζεται η επόμενη άφιξη. Αν ο server είναι ελεύθερος, η νέα εργασία τύπου 1 ξεκινά άμεσα εξυπηρέτηση. Διαφορετικά, τοποθετείται στο τέλος της ουράς.

Στο γεγονός DEPART1 καταγράφεται ο χρόνος απόκρισης της πρώτης φάσης και δημιουργείται νέα εργασία τύπου 2, η οποία προστίθεται στο τέλος της FIFO ουράς. Στη συνέχεια, ο server επιλέγει την επόμενη εργασία από την κεφαλή της ουράς.

Στο γεγονός DEPART2 καταγράφεται ο χρόνος απόκρισης της δεύτερης φάσης, αυξάνεται ο αριθμός των πλήρως εξυπηρετημένων requests και, αν υπάρχουν άλλες εργασίες

στην ουρά, ξεκινά η επόμενη εξυπηρέτηση. Διαφορετικά, ο server γίνεται idle.

Επειδή το μέγεθος της ουράς μεταβάλλεται μόνο στις χρονικές στιγμές γεγονότων, οι μέσοι αριθμοί εργασιών στο σύστημα πρέπει να υπολογίζονται ως χρονικά σταθμισμένοι μέσοι όροι. Αν $Q(t)$ είναι βηματική συνάρτηση, τότε:

$$\bar{Q} = \frac{\sum_i Q(t_i) \Delta t_i}{\sum_i \Delta t_i}, \quad \Delta t_i = t_{i+1} - t_i.$$

Ο απλός αριθμητικός μέσος των τιμών του $Q(t)$ στα event times δεν είναι σωστός, διότι τα γεγονότα δεν απέχουν ισοχρονικά μεταξύ τους.

1.3 Ερώτημα A.1: Εκτέλεση για $\lambda = 100$ requests/sec

Στο πρώτο ερώτημα εκτελέσαμε την προσομοίωση για:

$$\lambda = 100 \text{ requests/sec,}$$

μέχρι να εξυπηρετηθούν πλήρως 10^4 requests. Η εκτέλεση έδωσε τα ακόλουθα βασικά αριθμητικά αποτελέσματα:

A.1 $\lambda = 100$ req/sec, 10000 fully served requests

$t_{\text{end}} = 100837.637255 \text{ ms} = 100.837637 \text{ sec}$

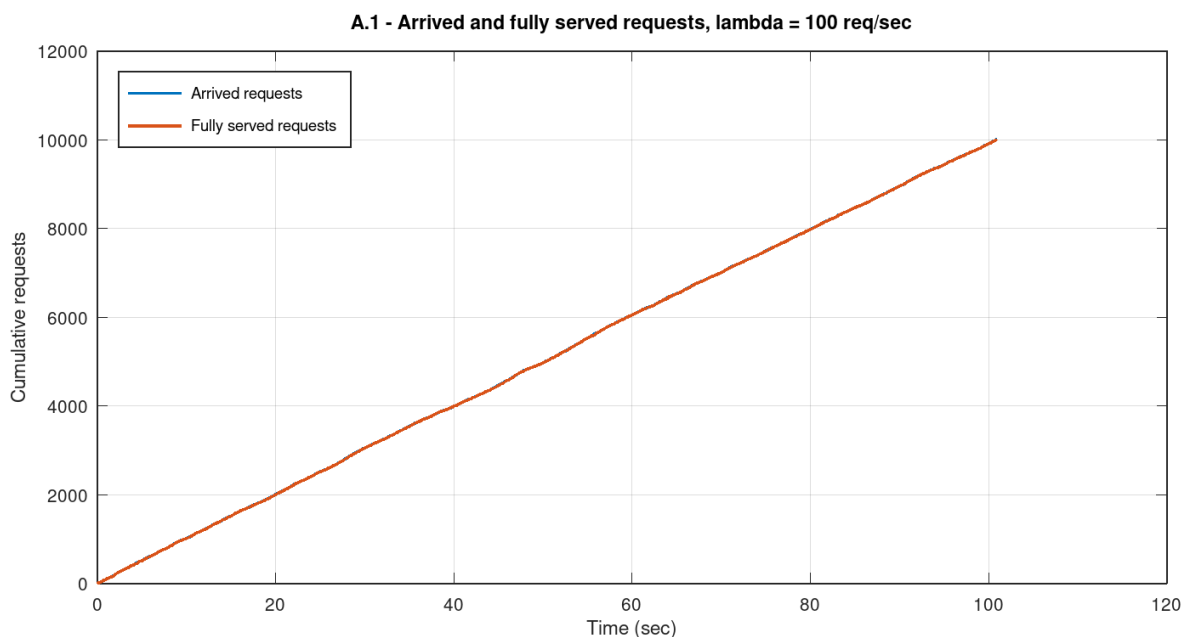
arrivals = 10002, fully served = 10000

$R_1 = 10.579663 \text{ ms}$, $R_2 = 6.398254 \text{ ms}$, $R = 16.977038 \text{ ms}$

throughput: type-1 = 99.179238 jobs/sec, type-2 = 99.169321 jobs/sec

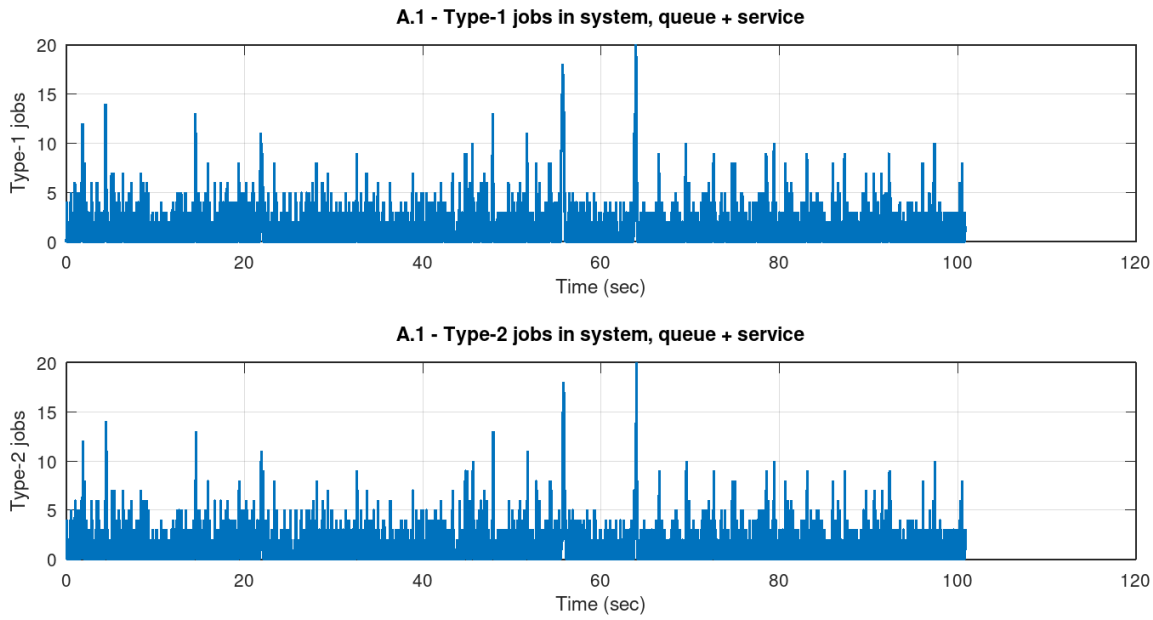
time-weighted mean jobs: $N_1 = 1.049398$, $N_2 = 0.634528$, $N = 1.683926$

Στο Σχήμα 1 παρουσιάζονται οι αθροιστικοί αριθμοί αφίξεων και πλήρως εξυπηρετημένων requests ως προς τον χρόνο. Οι δύο καμπύλες είναι πρακτικά σχεδόν ταυτισμένες. Αυτό είναι αναμενόμενο, επειδή για $\lambda = 100$ το σύστημα είναι στάσιμο και δεν συσσωρεύει απεριόριστα μεγάλο πλήθος εργασιών.



Σχήμα 1: Αθροιστικός αριθμός αφίξεων και πλήρως εξυπηρετημένων requests για $\lambda = 100$ requests/sec.

Στο Σχήμα 2 φαίνεται η εξέλιξη του πλήθους εργασιών τύπου 1 και τύπου 2 στο σύστημα, όπου ως σύστημα θεωρούμε τόσο την ουρά όσο και την εργασία που ενδεχομένως εξυπηρετείται εκείνη τη στιγμή. Παρατηρούμε ότι και τα δύο μεγέθη παρουσιάζουν τυχαίες διακυμάνσεις, αλλά δεν εμφανίζουν αυξητική τάση ως προς τον χρόνο. Αυτό είναι ακόμη μία ένδειξη στάσιμης λειτουργίας.



Σχήμα 2: Αριθμός εργασιών τύπου 1 και τύπου 2 στο σύστημα για $\lambda = 100$ requests/sec.

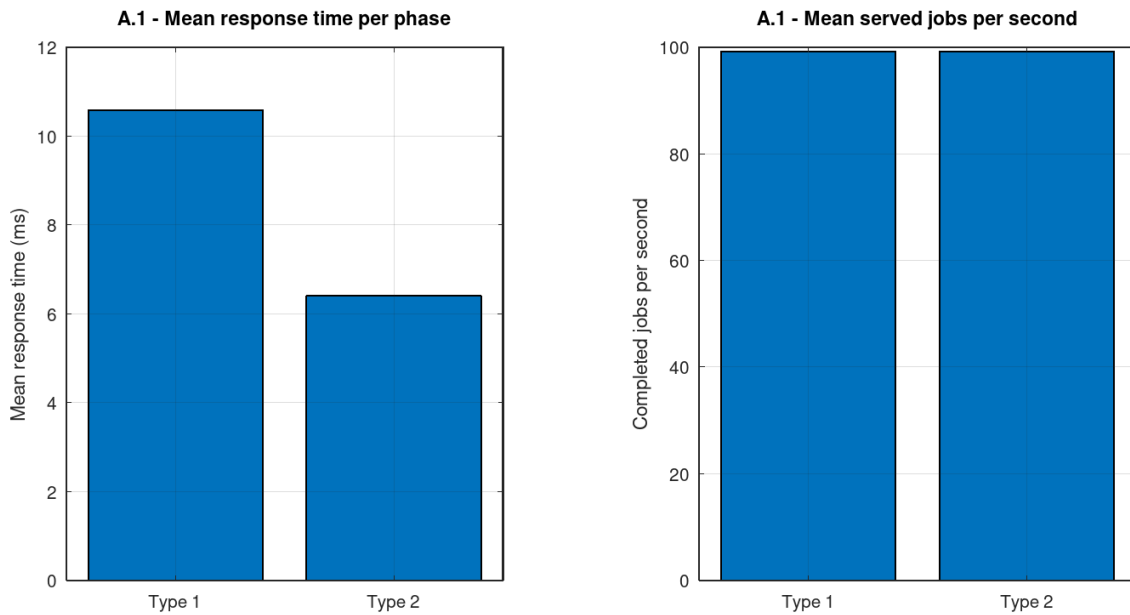
Στο Σχήμα 3 παρουσιάζονται οι μέσοι χρόνοι απόκρισης ανά φάση και οι αντίστοιχοι ρυθμοί ολοκλήρωσης εργασιών ανά δευτερόλεπτο. Ο μέσος χρόνος απόκρισης της εργασίας τύπου 1 είναι μεγαλύτερος από αυτόν της εργασίας τύπου 2, κάτι που συμφωνεί με το γεγονός ότι η εργασία τύπου 1 έχει σημαντικά μεγαλύτερο μέσο χρόνο εξυπηρέτησης:

$$E[S_1] \simeq 4.481689 \text{ ms} \quad \text{έναντι} \quad E[S_2] = 0.75 \text{ ms}.$$

Επιπλέον, οι δύο ρυθμοί ολοκλήρωσης είναι σχεδόν ίσοι και κοντά στο $\lambda = 100$, διότι κάθε πλήρως εξυπηρετημένο request αντιστοιχεί σε μία εργασία τύπου 1 και μία εργασία τύπου 2.

Αριθμητικά, πήραμε:

$$R_1 = 10.579663 \text{ ms}, \quad R_2 = 6.398254 \text{ ms},$$



Σχήμα 3: Μέσοι χρόνοι απόκρισης ανά φάση και μέσος αριθμός ολοκληρωμένων εργασιών ανά δευτερόλεπτο.

και:

$$R = 16.977038 \text{ ms.}$$

Οι τιμές αυτές είναι μεγαλύτερες από τους καθαρούς μέσους χρόνους εξυπηρέτησης, επειδή περιλαμβάνουν τόσο εξυπηρέτηση όσο και αναμονή στην ουρά. Αυτό είναι ιδιαίτερα σημαντικό: το R_i δεν είναι απλώς service time, αλλά response time της αντίστοιχης φάσης.

1.4 Ερώτημα Α.2: Στασιμότητα για διαφορετικές τιμές του λ

Στο δεύτερο ερώτημα εκτελέσαμε την προσομοίωση για:

$$\lambda \in \{50, 100, 150, 200, 250\} \text{ requests/sec.}$$

Το αναλυτικό κριτήριο στασιμότητας προκύπτει από το συνολικό μέσο έργο που εισάγει κάθε request στον server. Κάθε request απαιτεί κατά μέσο όρο:

$$E[S_1] + E[S_2] = e^{1.5} + 0.75 \simeq 5.231689 \text{ ms.}$$

Άρα ο βαθμός φόρτου του single-server συστήματος είναι:

$$\rho = \frac{\lambda(E[S_1] + E[S_2])}{1000}.$$

Για να είναι το σύστημα στάσιμο, πρέπει:

$$\rho < 1.$$

Επομένως:

$$\lambda < \frac{1000}{E[S_1] + E[S_2]} = \frac{1000}{5.231689} \simeq 191.142858 \text{ requests/sec.}$$

Η έξοδος του προγράμματος για το ερώτημα αυτό ήταν:

A.2 Stability bound

```
lambda < 191.142858 req/sec
lambda = 50: rho = 0.261584 -> stationary
lambda = 100: rho = 0.523169 -> stationary
lambda = 150: rho = 0.784753 -> stationary
lambda = 200: rho = 1.046338 -> non-stationary
lambda = 250: rho = 1.307922 -> non-stationary
```

Άρα, αναλυτικά, οι τιμές:

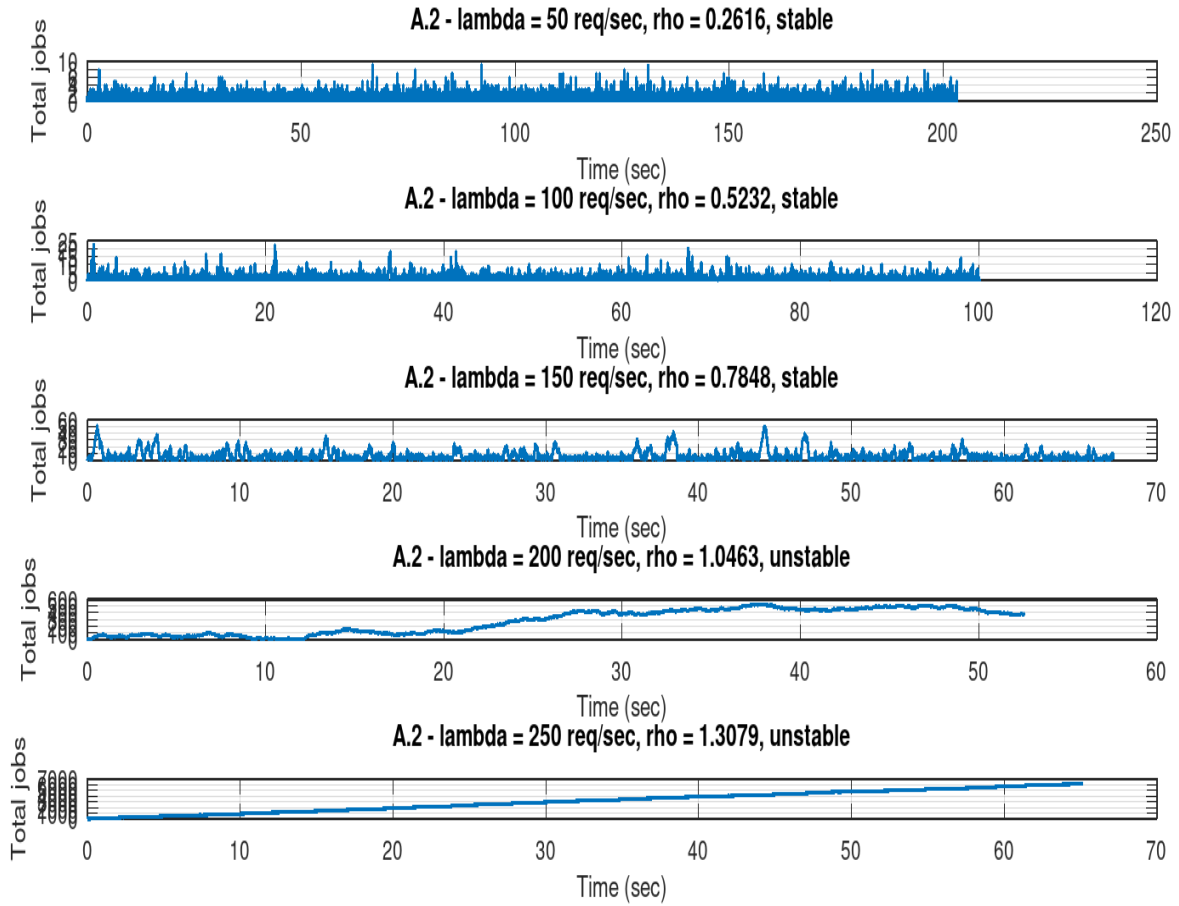
$$\lambda = 50, 100, 150$$

αντιστοιχούν σε στάσιμη λειτουργία, ενώ οι τιμές:

$$\lambda = 200, 250$$

αντιστοιχούν σε μη στάσιμη λειτουργία.

Στο Σχήμα 4 φαίνεται το συνολικό πλήθος εργασιών στο σύστημα ως συνάρτηση του χρόνου για τις πέντε τιμές του λ . Για $\lambda = 50, 100, 150$, το πλήθος εργασιών εμφανίζει διακυμάνσεις γύρω από κάποια πεπερασμένη περιοχή τιμών, χωρίς συστηματική ανοδική τάση. Αντίθετα, για $\lambda = 200$ και κυρίως για $\lambda = 250$, παρατηρούμε καθαρή συσσώρευση εργασιών, κάτι που συμφωνεί με το γεγονός ότι $\rho > 1$.



Σχήμα 4: Συνολικό πλήθος εργασιών στο σύστημα για $\lambda \in \{50, 100, 150, 200, 250\}$ requests/sec.

Η συμπεριφορά αυτή συμφωνεί πλήρως με τη θεωρία. Όταν $\rho < 1$, ο server μπορεί κατά μέσο όρο να επεξεργάζεται το εισερχόμενο έργο και η ουρά έχει στάσιμη κατανομή. Όταν $\rho \geq 1$, ο μέσος ρυθμός εισερχόμενου έργου είναι μεγαλύτερος ή ίσος από τη δυνατότητα εξυπηρέτησης του server, με αποτέλεσμα το σύστημα να μην έχει στάσιμη κατάσταση και η ουρά να αυξάνεται μακροπρόθεσμα.

1.5 Ερώτημα Α.3: Διαστήματα εμπιστοσύνης και αφαίρεση μεταβατικής περιόδου

Στο τρίτο ερώτημα εκτελέσαμε:

$$n = 30$$

ανεξάρτητες επαναλήψεις της προσομοίωσης για:

$$\lambda = 100 \quad \text{και} \quad \lambda = 180 \text{ requests/sec.}$$

Κάθε επανάληψη δίνει έναν χρονικά σταθμισμένο μέσο αριθμό εργασιών τύπου 1 και τύπου 2 στο σύστημα. Τα 30 αυτά ανεξάρτητα μεγέθη χρησιμοποιούνται για τον υπολογισμό διαστημάτων εμπιστοσύνης 95%.

Η χρήση ανεξάρτητων επαναλήψεων είναι απαραίτητη, επειδή οι τιμές $Q(t)$ μέσα σε ένα μόνο simulation run δεν αποτελούν ανεξάρτητα δείγματα. Αν η ουρά έχει μία συγκεκριμένη τιμή σε κάποια στιγμή, είναι πολύ πιθανό να έχει κοντινή τιμή και αμέσως μετά. Άρα, για τα confidence intervals δεν χρησιμοποιούμε τις διαδοχικές τιμές της ουράς, αλλά έναν συνοπτικό μέσο όρο από κάθε ανεξάρτητη εκτέλεση.

Για το διάστημα εμπιστοσύνης της μέσης τιμής χρησιμοποιούμε:

$$\bar{x} \pm t_{n-1, 1-\alpha/2} \frac{s}{\sqrt{n}}, \quad \alpha = 0.05.$$

Για τη διάμεσο χρησιμοποιούμε διάστημα βασισμένο σε στατιστικές τάξης:

$$[X_{(k)}, X_{(n+1-k)}],$$

με:

$$k = \left\lfloor \frac{n}{2} - z_{1-\alpha/2} \frac{\sqrt{n}}{2} \right\rfloor.$$

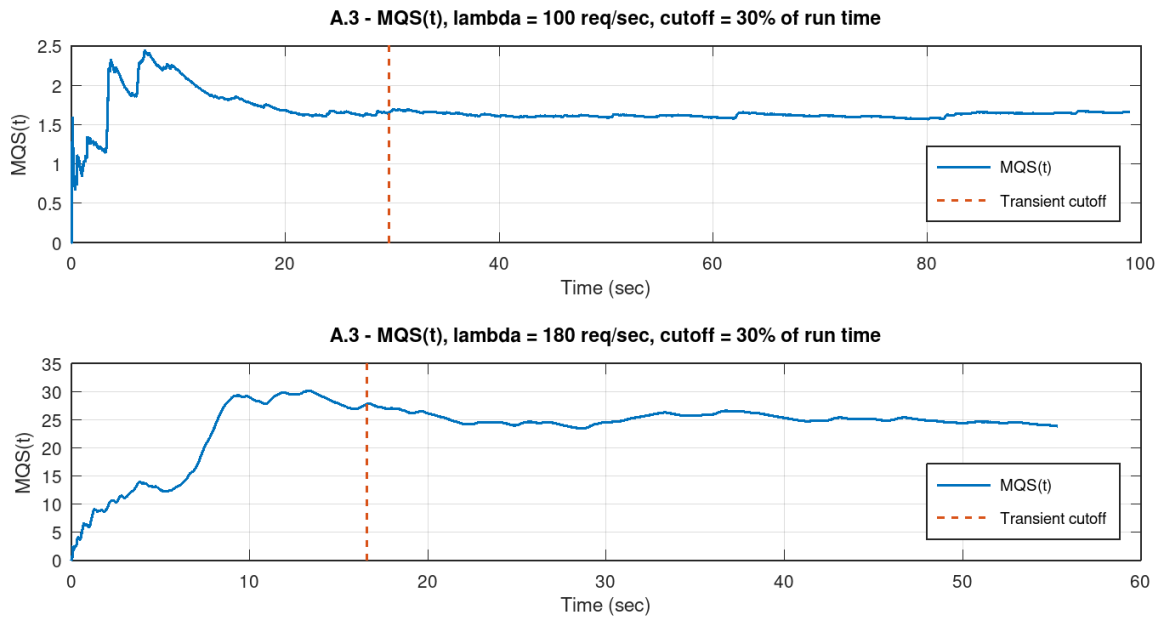
1.5.1 Μέθοδος αφαίρεσης transient

Επειδή η προσομοίωση ξεκινά από άδαιο σύστημα, το αρχικό τμήμα δεν είναι απαραίτητα αντιπροσωπευτικό της μόνιμης κατάστασης. Για τον λόγο αυτό χρησιμοποιήσαμε τη μέθοδο MQS, δηλαδή το running time-weighted mean queue size:

$$MQS(t) = \frac{1}{t} \int_0^t Q(s) ds.$$

Η βασική ιδέα είναι ότι το $MQS(t)$ αρχικά επηρεάζεται έντονα από την αρχική κατάσταση, αλλά στη συνέχεια τείνει να σταθεροποιείται γύρω από τη μέση τιμή μόνιμης λειτουργίας. Στην υλοποίηση επιλέχθηκε ως cutoff το πρώτο 30% του χρόνου κάθε run, επιλογή που επιβεβαιώνεται και από τα MQS διαγράμματα.

Στο Σχήμα 5 παρουσιάζεται το $MQS(t)$ για $\lambda = 100$ και $\lambda = 180$. Για $\lambda = 100$ παρατηρούμε ότι μετά τα πρώτα δευτερόλεπτα η καμπύλη σταθεροποιείται γρήγορα. Για $\lambda = 180$, επειδή το σύστημα λειτουργεί κοντά στο όριο στασιμότητας, η καμπύλη εμφανίζει πολύ μεγαλύτερο transient και υψηλότερη μεταβλητότητα.



Σχήμα 5: $MQS(t)$ για $\lambda = 100$ και $\lambda = 180$ requests/sec και επιλογή cutoff στο 30% του run time.

Για $\lambda = 180$ ο βαθμός φόρτου είναι:

$$\rho_{180} = \frac{180 \cdot 5.231689}{1000} \simeq 0.9417.$$

Άρα το σύστημα είναι μεν στάσιμο, αλλά βρίσκεται πολύ κοντά στο όριο $\rho = 1$. Αυτό εξηγεί γιατί τα confidence intervals είναι πολύ πλατύτερα σε σχέση με την περίπτωση $\lambda = 100$.

1.5.2 Αποτελέσματα διαστημάτων εμπιστοσύνης

Η έξοδος του προγράμματος για τα confidence intervals ήταν:

A.3 95% confidence intervals, $n = 30$ replications

Transient removal rule: remove first 30% of each run, guided by MQS(t) plots.

$\lambda = 100$ req/sec

Full trace:

type 1: mean CI [1.014991, 1.056171], median CI [1.011125, 1.063644]

type 2: mean CI [0.603187, 0.632896], median CI [0.601051, 0.631790]

After transient removal:

type 1: mean CI [1.008903, 1.060815], median CI [0.984703, 1.060240]

type 2: mean CI [0.599920, 0.636938], median CI [0.586471, 0.647216]

$\lambda = 180$ req/sec

Full trace:

type 1: mean CI [14.476681, 18.489433], median CI [13.566229, 18.068241]

type 2: mean CI [13.679712, 17.688760], median CI [12.826022, 17.198553]

After transient removal:

type 1: mean CI [14.638395, 19.446088], median CI [13.488622, 18.944785]

type 2: mean CI [13.891899, 18.707440], median CI [12.648005, 18.337582]

Για $\lambda = 100$, τα διαστήματα εμπιστοσύνης είναι σχετικά στενά. Αυτό είναι αναμενόμενο, επειδή το σύστημα έχει μέτριο φορτίο:

$$\rho_{100} \simeq 0.5232.$$

Άρα οι διακυμάνσεις του πλήθους εργασιών είναι περιορισμένες και οι ανεξάρτητες επαναλήψεις παράγουν σχετικά κοντινές εκτιμήσεις.

Για $\lambda = 180$, τα διαστήματα εμπιστοσύνης είναι πολύ πλατύτερα. Αυτό συμβαίνει επειδή το σύστημα βρίσκεται κοντά στο όριο αστάθειας. Σε τέτοιες περιπτώσεις, μικρές τυχαίες

αποκλίσεις στην ακολουθία αφίξεων και εξυπηρετήσεων μπορούν να οδηγήσουν σε μεγάλες παροδικές ουρές, άρα οι εκτιμήσεις από run σε run διαφέρουν περισσότερο.

Η αφαίρεση του transient δεν αλλάζει δραματικά τα κέντρα των διαστημάτων, αλλά μεταβάλλει ελαφρώς τόσο τα άκρα όσο και το πλάτος τους. Για $\lambda = 100$ η επίδραση είναι μικρή, επειδή το σύστημα φτάνει γρήγορα σε στάσιμη λειτουργία. Για $\lambda = 180$ η επίδραση είναι πιο αισθητή, επειδή η αρχική περίοδος και η αργότερη σύγκλιση της ουράς έχουν μεγαλύτερο ρόλο.

Συνολικά, τα αποτελέσματα επιβεβαιώνουν ότι όσο πιο κοντά βρίσκεται το σύστημα στο $\rho = 1$, τόσο περισσότερες επαναλήψεις ή/και μεγαλύτερη διάρκεια προσομοίωσης απαιτούνται για πιο στενά και αξιόπιστα confidence intervals.

1.6 Ερώτημα Α.4: Επαλήθευση του νόμου Little

Στο τέταρτο ερώτημα επαληθεύουμε τον νόμο του Little:

$$N = \lambda R.$$

Στη συγκεκριμένη προσομοίωση το R είναι ο μέσος end-to-end χρόνος απόκρισης ενός request, δηλαδή από την άφιξή του μέχρι την ολοκλήρωση της δεύτερης φάσης. Επομένως:

$$R = R_1 + R_2.$$

Το N είναι ο χρονικά σταθμισμένος μέσος αριθμός εργασιών στο σύστημα:

$$N = \overline{Q_1 + Q_2}.$$

Χρειάζεται ιδιαίτερη προσοχή στις μονάδες. Το λ δίνεται σε requests/sec, ενώ το R έχει υπολογιστεί σε milliseconds. Άρα χρησιμοποιούμε:

$$\lambda_{ms} = \frac{\lambda}{1000}.$$

Η έξοδος του προγράμματος για τον νόμο Little ήταν:

A.4 Little's Law, lambda = 100 req/sec

Without transient removal:

R = 16.977038 ms, N_sim = 1.683926, lambda*R = 1.697704, error = 0.818167 %

With transient removal:

R = 16.830559 ms, N_sim = 1.651705, lambda*R = 1.683056, error = 1.898066 %

Χωρίς αφαίρεση transient έχουμε:

$$R = 16.977038 \text{ ms},$$

και:

$$\lambda_{ms} R = 0.1 \cdot 16.977038 = 1.697704.$$

Η προσομοίωση δίνει:

$$N_{\text{sim}} = 1.683926.$$

Το σχετικό σφάλμα είναι:

$$0.818167\%.$$

Άρα ο νόμος Little επαληθεύεται με πολύ καλή ακρίβεια.

Μετά την αφαίρεση του transient, έχουμε:

$$R = 16.830559 \text{ ms},$$

και:

$$\lambda_{ms} R = 1.683056.$$

Το αντίστοιχο χρονικά σταθμισμένο μέσο πλήθος εργασιών είναι:

$$N_{\text{sim}} = 1.651705,$$

με σχετικό σφάλμα:

$$1.898066\%.$$

Το γεγονός ότι το σφάλμα μετά την αφαίρεση transient είναι λίγο μεγαλύτερο δεν αποτελεί πρόβλημα. Ο νόμος Little ισχύει ακριβώς στη μόνιμη κατάσταση και στο όριο μεγάλου χρόνου, ενώ εδώ χρησιμοποιείται ένα πεπερασμένο single-run δείγμα και ένα συγκεκριμένο cutoff. Επιπλέον, μετά το cutoff χρησιμοποιούνται λιγότερες παρατηρήσεις, άρα είναι φυσιολογικό να εμφανιστεί ελαφρώς μεγαλύτερη δειγματική απόκλιση.

Το σημαντικό συμπέρασμα είναι ότι και στις δύο περιπτώσεις το σφάλμα είναι μικρό, κάτω από 2%, γεγονός που επιβεβαιώνει ότι η προσομοίωση είναι συνεπής με τη βασική θεωρία των συστημάτων αναμονής.

1.7 Συνολικό συμπέρασμα πρώτου μέρους

Τα αποτελέσματα του πρώτου μέρους είναι συνεπή τόσο με τη θεωρία όσο και με τη φυσική ερμηνεία του συστήματος.

Αρχικά, για $\lambda = 100 \text{ requests/sec}$, το σύστημα λειτουργεί σε σαφώς στάσιμη περιοχή, αφού:

$$\rho = 0.523169 < 1.$$

Αυτό φαίνεται άμεσα από την σχεδόν πλήρη ταύτιση των αθροιστικών αφίξεων και εξυπηρετήσεων, καθώς και από το ότι οι καμπύλες των εργασιών τύπου 1 και τύπου 2 δεν εμφανίζουν ανοδική μακροχρόνια τάση. Ο μέσος end-to-end χρόνος απόκρισης είναι περίπου 16.98 ms, ενώ ο χρονικά σταθμισμένος μέσος αριθμός εργασιών στο σύστημα είναι περίπου 1.68.

Στη συνέχεια, η ανάλυση της στασιμότητας δείχνει ότι το κρίσιμο όριο είναι:

$$\lambda_{\max} \simeq 191.142858 \text{ requests/sec.}$$

Άρα οι τιμές $\lambda = 50, 100, 150$ είναι στάσιμες, ενώ οι τιμές $\lambda = 200, 250$ δεν είναι στάσιμες. Τα αντίστοιχα διαγράμματα επιβεβαιώνουν πλήρως το αναλυτικό αποτέλεσμα: στα στάσιμα σενάρια η ουρά παραμένει σε πεπερασμένες διακυμάνσεις, ενώ στα μη στάσιμα εμφανίζει αυξητική τάση.

Τα confidence intervals δείχνουν επίσης την αναμενόμενη εξάρτηση από το φορτίο. Για $\lambda = 100$ είναι στενά, ενώ για $\lambda = 180$ γίνονται πολύ πλατύτερα, επειδή το σύστημα βρίσκεται κοντά στο όριο $\rho = 1$. Αυτό είναι ένα κλασικό χαρακτηριστικό των ουρών: όσο το φορτίο πλησιάζει τη μονάδα, τόσο αυξάνεται η μεταβλητότητα, οι ουρές γίνονται πιο έντονες και η προσομοίωση χρειάζεται περισσότερες παρατηρήσεις για σταθερές εκτιμήσεις.

Η μέθοδος MQS αποτελεί εύλογο εργαλείο για την επιλογή transient cutoff. Τα διαγράμματα δείχνουν ότι για $\lambda = 100$ το running time-weighted average σταθεροποιείται γρήγορα, ενώ για $\lambda = 180$ η σταθεροποίηση είναι πιο αργή και η καμπύλη πιο ευμετάβλητη. Η επιλογή αποκοπής του πρώτου 30% του run είναι επομένως συντηρητική και κατάλληλη για την παρούσα άσκηση.

Τέλος, ο νόμος του Little επαληθεύεται με μικρό σφάλμα τόσο πριν όσο και μετά την αφαίρεση transient. Αυτό λειτουργεί ως επιπλέον έλεγχος ορθότητας, επειδή συνδέει τρεις ποσότητες που υπολογίζονται με διαφορετικό τρόπο: τον ρυθμό αφίξεων, τον μέσο χρόνο απόκρισης και τον χρονικά σταθμισμένο μέσο αριθμό εργασιών στο σύστημα.

Συνολικά, μπορούμε να συμπεράνουμε ότι η προσομοίωση είναι σωστή. Τα διαγράμματα συμφωνούν με τα αναλυτικά κριτήρια στασιμότητας, τα confidence intervals έχουν λογική συμπεριφορά, και ο νόμος Little επιβεβαιώνεται με μικρή απόκλιση.

2 Ανοιχτό δίκτυο ουρών αναμονής

2.1 Περιγραφή του δικτύου και παραδοχές Jackson

Στο δεύτερο μέρος της άσκησης μελετάμε ένα ανοιχτό δίκτυο ουρών αναμονής με πέντε ουρές:

$$Q_1, Q_2, Q_3, Q_4, Q_5.$$

Οι εξωτερικές αφίξεις εισέρχονται στο δίκτυο με ρυθμούς λ_1 και λ_2 , ενώ κάθε ουρά Q_i διαθέτει έναν εξυπηρετητή με εκθετικά κατανομημένους χρόνους εξυπηρέτησης και ρυθμό μ_i .

Για να μπορεί το δίκτυο να μελετηθεί ως ανοιχτό δίκτυο Jackson, πρέπει να ισχύουν οι βασικές παραδοχές:

- οι εξωτερικές αφίξεις είναι Poisson,
- οι χρόνοι εξυπηρέτησης σε κάθε ουρά είναι εκθετικά κατανομημένοι,
- οι εξυπηρετήσεις είναι ανεξάρτητες μεταξύ τους και ανεξάρτητες από τις αφίξεις,
- μετά από κάθε εξυπηρέτηση, ο πελάτης δρομολογείται με σταθερές πιθανότητες σε επόμενη ουρά ή αποχωρεί από το δίκτυο,
- οι πιθανότητες δρομολόγησης δεν εξαρτώνται από την κατάσταση του δικτύου,
- κάθε ουρά έχει άπειρη χωρητικότητα,
- κάθε κόμβος μπορεί να θεωρηθεί ως ξεχωριστό σύστημα $M/M/1$ με δικό του συνολικό ρυθμό άφιξης,
- το δίκτυο είναι εργοδικό, δηλαδή για κάθε ουρά ισχύει $\rho_i < 1$.

Το θεώρημα Jackson επιτρέπει, υπό τις παραπάνω παραδοχές, τη μελέτη του δικτύου ως γινόμενο ανεξάρτητων ουρών $M/M/1$. Συνεπώς, αφού βρούμε τους συνολικούς ρυθμούς άφιξης Γ_i σε κάθε κόμβο, μπορούμε να υπολογίσουμε:

$$\rho_i = \frac{\Gamma_i}{\mu_i}, \quad E[N_i] = \frac{\rho_i}{1 - \rho_i}.$$

2.2 Μεταβάσεις και πιθανότητες δρομολόγησης

Από το σχήμα του δικτύου προκύπτουν οι εξής δρομολογήσεις:

Από	Προς	Πιθανότητα
Q_1	Q_2	$\frac{1}{2}$
Q_1	Q_3	$\frac{1}{4}$
Q_1	Έξοδος	$\frac{1}{4}$
Q_2	Q_3	$\frac{1}{3}$
Q_2	Q_4	$\frac{2}{3}$
Q_3	Q_5	1
Q_4	Q_5	$\frac{3}{5}$
Q_4	Έξοδος	$\frac{2}{5}$
Q_5	Έξοδος	1

Οι πιθανότητες $Q_1 \rightarrow Q_3$ και $Q_2 \rightarrow Q_4$ προκύπτουν ως συμπληρωματικές πιθανότητες των αντίστοιχων διακλαδώσεων. Συγκεκριμένα:

$$P(Q_1 \rightarrow Q_3) = 1 - \frac{1}{2} - \frac{1}{4} = \frac{1}{4},$$

και:

$$P(Q_2 \rightarrow Q_4) = 1 - \frac{1}{3} = \frac{2}{3}.$$

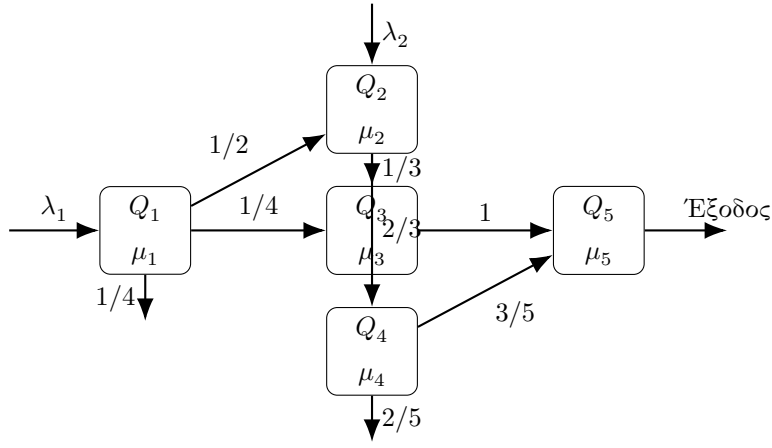
Για πληρότητα, στο Σχήμα 6 δίνεται σχηματική αναπαράσταση του δικτύου με τις βασικές πιθανότητες δρομολόγησης.

2.3 Ερώτημα Β.2: Εξισώσεις ροής και εντάσεις φορτίου

Για την εφαρμογή του θεωρήματος Jackson πρέπει πρώτα να βρούμε τον συνολικό ρυθμό άφιξης σε κάθε ουρά. Συμβολίζουμε αυτούς τους ρυθμούς με Γ_i , ώστε να μη συγχέονται με τους εξωτερικούς ρυθμούς λ_1 και λ_2 .

Από τις δρομολογήσεις του σχήματος έχουμε:

$$\Gamma_1 = \lambda_1,$$



Σχήμα 6: Σχηματική αναπαράσταση του ανοιχτού δικτύου ουρών Jackson.

$$\Gamma_2 = \lambda_2 + \frac{1}{2}\Gamma_1,$$

$$\Gamma_3 = \frac{1}{4}\Gamma_1 + \frac{1}{3}\Gamma_2,$$

$$\Gamma_4 = \frac{2}{3}\Gamma_2,$$

$$\Gamma_5 = \Gamma_3 + \frac{3}{5}\Gamma_4.$$

Αν αντικαταστήσουμε διαδοχικά, παίρνουμε όλες τις Γ_i ως συναρτήσεις μόνο των εξωτερικών ρυθμών λ_1, λ_2 :

$$\Gamma_1 = \lambda_1,$$

$$\Gamma_2 = \lambda_2 + \frac{1}{2}\lambda_1,$$

$$\Gamma_3 = \frac{1}{4}\lambda_1 + \frac{1}{3}\left(\lambda_2 + \frac{1}{2}\lambda_1\right) = \frac{5}{12}\lambda_1 + \frac{1}{3}\lambda_2,$$

$$\Gamma_4 = \frac{2}{3}\left(\lambda_2 + \frac{1}{2}\lambda_1\right) = \frac{1}{3}\lambda_1 + \frac{2}{3}\lambda_2,$$

και:

$$\begin{aligned} \Gamma_5 &= \Gamma_3 + \frac{3}{5}\Gamma_4 \\ &= \left(\frac{5}{12}\lambda_1 + \frac{1}{3}\lambda_2\right) + \frac{3}{5}\left(\frac{1}{3}\lambda_1 + \frac{2}{3}\lambda_2\right) \\ &= \frac{37}{60}\lambda_1 + \frac{11}{15}\lambda_2. \end{aligned}$$

Επομένως, οι εντάσεις φορτίου των πέντε ουρών είναι:

$$\rho_1 = \frac{\lambda_1}{\mu_1},$$

$$\begin{aligned}\rho_2 &= \frac{\lambda_2 + \frac{1}{2}\lambda_1}{\mu_2}, \\ \rho_3 &= \frac{\frac{5}{12}\lambda_1 + \frac{1}{3}\lambda_2}{\mu_3}, \\ \rho_4 &= \frac{\frac{1}{3}\lambda_1 + \frac{2}{3}\lambda_2}{\mu_4}, \\ \rho_5 &= \frac{\frac{37}{60}\lambda_1 + \frac{11}{15}\lambda_2}{\mu_5}.\end{aligned}$$

Η συνθήκη εργοδικότητας του δικτύου είναι:

$$\rho_i < 1, \quad i = 1, \dots, 5.$$

Αν έστω μία ουρά έχει $\rho_i \geq 1$, τότε το δίκτυο δεν είναι εργοδικό.

Η συνάρτηση `intensities` που υλοποιήθηκε στο Octave υπολογίζει ακριβώς αυτές τις τιμές και επιστρέφει επίσης μία δυαδική ένδειξη εργοδικότητας.

2.4 Ερώτημα B.3: Μέσος αριθμός πελατών σε κάθε ουρά

Κάθε ουρά του δικτύου, υπό τις παραδοχές Jackson, μπορεί να μελετηθεί ως ξεχωριστή ουρά $M/M/1$ με συνολικό ρυθμό άφιξης Γ_i και ρυθμό εξυπηρέτησης μ_i . Για μία ουρά $M/M/1$, όταν $\rho_i < 1$, ο μέσος αριθμός πελατών στο σύστημα είναι:

$$E[N_i] = \frac{\rho_i}{1 - \rho_i}.$$

Άρα, αφού υπολογιστούν οι εντάσεις φορτίου ρ_i , οι μέσοι αριθμοί πελατών υπολογίζονται άμεσα από τον παραπάνω τύπο. Η συνάρτηση `mean_clients` καλεί πρώτα την `intensities`, ελέγχει την εργοδικότητα, και στη συνέχεια επιστρέφει το διάνυσμα:

$$[E[N_1], E[N_2], E[N_3], E[N_4], E[N_5]].$$

2.5 Ερώτημα B.4: Αριθμητική εφαρμογή για τις δοθείσες παραμέτρους

Για το αριθμητικό ερώτημα της εκφώνησης δίνονται:

$$\lambda_1 = 10, \quad \lambda_2 = 6,$$

και:

$$\mu_1 = 15, \quad \mu_2 = 20, \quad \mu_3 = 10, \quad \mu_4 = 14, \quad \mu_5 = 12.$$

Όλοι οι ρυθμοί είναι σε πελάτες ανά δευτερόλεπτο.

Η εκτέλεση του Octave έδωσε τους παρακάτω effective arrival rates:

$$\Gamma = [10, 11, 6.1666666667, 7.3333333333, 10.5666666667].$$

Οι αντίστοιχες εντάσεις φορτίου είναι:

$$\rho = [0.6666666667, 0.55, 0.6166666667, 0.5238095238, 0.8805555556].$$

Εφόσον όλες οι τιμές είναι μικρότερες της μονάδας, το δίκτυο είναι εργοδικό.

Οι μέσοι αριθμοί πελατών στις ουρές είναι:

$$E[N_i] = [2.0000000000, 1.2222222222, 1.6086956522, 1.1000000000, 7.3720930233].$$

Άρα ο συνολικός μέσος αριθμός πελατών στο δίκτυο είναι:

$$\begin{aligned} E[N] &= \sum_{i=1}^5 E[N_i] \\ &= 13.3030108977. \end{aligned}$$

Ο συνολικός εξωτερικός ρυθμός εισόδου στο δίκτυο είναι:

$$\gamma = \lambda_1 + \lambda_2 = 16 \text{ πελάτες/sec.}$$

Επομένως, από τον νόμο του Little, ο μέσος end-to-end χρόνος καθυστέρησης ενός πελάτη στο δίκτυο είναι:

$$E[T] = \frac{E[N]}{\gamma} = \frac{13.3030108977}{16} = 0.8314381811 \text{ sec.}$$

Η έξοδος του προγράμματος για τα ερωτήματα B.2–B.4 ήταν:

Effective arrival rates Gamma_i:

Gamma1 = 10.0000000000 clients/sec

Gamma2 = 11.0000000000 clients/sec

Gamma3 = 6.1666666667 clients/sec

Gamma4 = 7.3333333333 clients/sec

Gamma5 = 10.5666666667 clients/sec

Traffic intensities $\rho_{i} = \Gamma_{i} / \mu_{i}$:

$$\rho_1 = 0.6666666667$$

$$\rho_2 = 0.5500000000$$

$$\rho_3 = 0.6166666667$$

$$\rho_4 = 0.5238095238$$

$$\rho_5 = 0.8805555556$$

Ergodicity flag: 1

The network is ergodic because all ρ_i are smaller than 1.

Mean clients $E[N_i]$ in each queue:

$$E[N_1] = 2.0000000000$$

$$E[N_2] = 1.2222222222$$

$$E[N_3] = 1.6086956522$$

$$E[N_4] = 1.1000000000$$

$$E[N_5] = 7.3720930233$$

Total mean number of clients in the network:

$$E[N] = \sum_i E[N_i] = 13.3030108977$$

End-to-end mean delay by Little's Law:

$$E[T] = E[N] / \gamma_{\text{external}} = 0.8314381811 \text{ seconds}$$

Παρατηρούμε ότι η μεγαλύτερη συνεισφορά στον συνολικό αριθμό πελατών προέρχεται από την ουρά Q_5 , όπου:

$$E[N_5] = 7.3720930233.$$

Αυτό είναι αναμενόμενο, επειδή η Q_5 έχει και τη μεγαλύτερη ένταση φορτίου:

$$\rho_5 = 0.8805555556.$$

Όσο το ρ_i πλησιάζει τη μονάδα, ο όρος $\rho_i / (1 - \rho_i)$ αυξάνεται απότομα. Επομένως, η Q_5

επηρεάζει δυσανάλογα τη συνολική καθυστέρηση του δικτύου.

2.6 Ερώτημα B.5: Στενωπός και μέγιστη τιμή του λ

Η στενωπός του δικτύου είναι η ουρά με τη μεγαλύτερη ένταση φορτίου ρ_i . Για τις δοθείσες τιμές, έχουμε:

$$\max_i \rho_i = \rho_5 = 0.8805555556.$$

Άρα η στενωπός του δικτύου είναι:

$$Q_5.$$

Στη συνέχεια ζητείται η μέγιστη τιμή της παραμέτρου λ_1 , ώστε το σύστημα να παραμένει εργοδικό, με σταθερό:

$$\lambda_2 = 6.$$

Για να είναι το δίκτυο εργοδικό πρέπει να ισχύει:

$$\Gamma_i < \mu_i, \quad i = 1, \dots, 5.$$

Από κάθε ουρά προκύπτει ένα άνω φράγμα για το λ_1 .

Για την Q_1 :

$$\Gamma_1 = \lambda_1 < \mu_1 = 15 \Rightarrow \lambda_1 < 15.$$

Για την Q_2 :

$$\Gamma_2 = 6 + \frac{1}{2}\lambda_1 < 20 \Rightarrow \lambda_1 < 28.$$

Για την Q_3 :

$$\Gamma_3 = \frac{5}{12}\lambda_1 + \frac{1}{3} \cdot 6 = \frac{5}{12}\lambda_1 + 2 < 10,$$

άρα:

$$\frac{5}{12}\lambda_1 < 8 \Rightarrow \lambda_1 < 19.2.$$

Για την Q_4 :

$$\Gamma_4 = \frac{1}{3}\lambda_1 + \frac{2}{3} \cdot 6 = \frac{1}{3}\lambda_1 + 4 < 14,$$

άρα:

$$\lambda_1 < 30.$$

Για την Q_5 :

$$\Gamma_5 = \frac{37}{60}\lambda_1 + \frac{11}{15} \cdot 6 = \frac{37}{60}\lambda_1 + 4.4 < 12.$$

Επομένως:

$$\frac{37}{60}\lambda_1 < 7.6$$

και:

$$\lambda_1 < \frac{7.6 \cdot 60}{37} = \frac{456}{37} = 12.3243243243.$$

Συγκεντρωτικά, τα άνω φράγματα είναι:

Ουρά	Φράγμα για λ_1
Q_1	15.0000000000
Q_2	28.0000000000
Q_3	19.2000000000
Q_4	30.0000000000
Q_5	12.3243243243

Το αυστηρότερο φράγμα είναι αυτό της Q_5 . Άρα:

$$\lambda_{1,\max} = 12.3243243243 \text{ πελάτες/sec.}$$

Η έξοδος του προγράμματος επιβεβαιώνει το αποτέλεσμα:

Current bottleneck queue, based on maximum rho_i:

```
bottleneck = Q5
```

```
max rho      = 0.8805555556
```

Upper bounds on lambda1 imposed by each queue:

```
From Q1: lambda1 < 15.0000000000
```

```
From Q2: lambda1 < 28.0000000000
```

```
From Q3: lambda1 < 19.2000000000
```

```
From Q4: lambda1 < 30.0000000000
```

```
From Q5: lambda1 < 12.3243243243
```

Maximum lambda1 for ergodicity with lambda2 fixed at 6.0000000000:

lambda1_max = 12.3243243243 clients/sec

limiting queue = Q5

Το αποτέλεσμα είναι συνεπές με την προηγούμενη παρατήρηση ότι η Q_5 έχει τη μεγαλύτερη ένταση φορτίου. Καθώς αυξάνεται το λ_1 , η Q_5 είναι η πρώτη ουρά που φτάνει στο όριο $\rho_i = 1$, άρα αυτή καθορίζει το μέγιστο επιτρεπτό λ_1 για εργοδικότητα.

2.7 Ερώτημα B.6: Διάγραμμα καθυστέρησης ως προς λ

Στο τελευταίο ερώτημα κρατάμε σταθερές τις παραμέτρους:

$$\lambda_2 = 6, \quad \mu_1 = 15, \mu_2 = 20, \mu_3 = 10, \mu_4 = 14, \mu_5 = 12,$$

και μεταβάλλουμε το λ_1 από 0.1 έως 0.99 της μέγιστης επιτρεπτής τιμής:

$$\lambda_1 \in [0.1\lambda_{1,\max}, 0.99\lambda_{1,\max}].$$

Δηλαδή:

$$\lambda_1 \in [1.2324324324, 12.2010810811].$$

Για κάθε τιμή του λ_1 , υπολογίζουμε ξανά:

$$E[N_i] = \frac{\rho_i}{1 - \rho_i},$$

και στη συνέχεια:

$$E[T] = \frac{\sum_{i=1}^5 E[N_i]}{\lambda_1 + \lambda_2}.$$

Το παραγόμενο διάγραμμα φαίνεται στο Σχήμα 7. Η διακεκομμένη κατακόρυφη γραμμή δείχνει την τιμή $\lambda_1 = 10$ της εκφώνησης, ενώ η δεύτερη κατακόρυφη γραμμή δείχνει το όριο $\lambda_{1,\max}$.

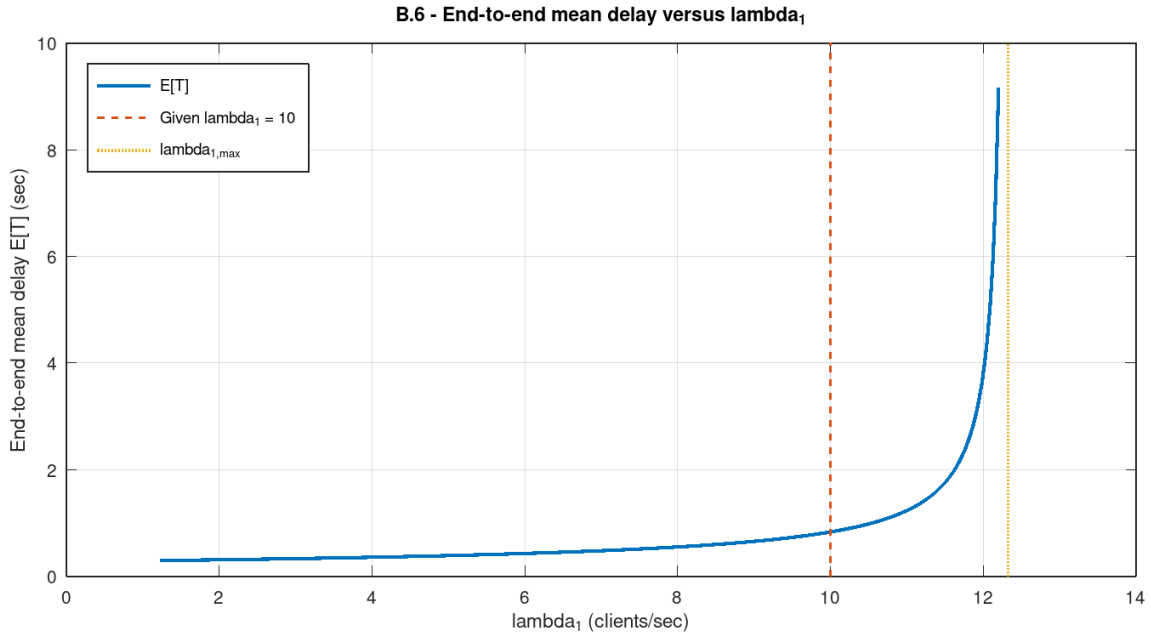
Η έξοδος του προγράμματος για επιλεγμένα σημεία ήταν:

Delay values at selected points:

lambda1 = 0.10 * lambda1_max = 1.2324324324 -> E[T] = 0.2950551294 sec

lambda1 = 0.25 * lambda1_max = 3.0810810811 -> E[T] = 0.3323554951 sec

lambda1 = 0.50 * lambda1_max = 6.1621621622 -> E[T] = 0.4349166146 sec



Σχήμα 7: Μέσος end-to-end χρόνος καθυστέρησης ως συνάρτηση του λ_1 .

$$\lambda_{d1} = 0.75 * \lambda_{d1,max} = 9.2432432432 \rightarrow E[T] = 0.6880326518 \text{ sec}$$

$$\lambda_{d1} = 0.90 * \lambda_{d1,max} = 11.0918918919 \rightarrow E[T] = 1.2973780957 \text{ sec}$$

$$\lambda_{d1} = 0.95 * \lambda_{d1,max} = 11.7081081081 \rightarrow E[T] = 2.2074842154 \text{ sec}$$

$$\lambda_{d1} = 0.99 * \lambda_{d1,max} = 12.2010810811 \rightarrow E[T] = 9.1519048705 \text{ sec}$$

Παρατηρούμε ότι για μικρές και μεσαίες τιμές του λ_1 ο μέσος χρόνος καθυστέρησης αυξάνεται σχετικά ομαλά. Όμως, όταν το λ_1 πλησιάζει το $\lambda_{1,max}$, η καθυστέρηση αυξάνεται απότομα. Αυτό οφείλεται στο γεγονός ότι η ουρά Q_5 πλησιάζει σε:

$$\rho_5 \rightarrow 1.$$

Καθώς:

$$E[N_5] = \frac{\rho_5}{1 - \rho_5},$$

όταν ρ_5 τείνει στη μονάδα, ο παρονομαστής $1 - \rho_5$ τείνει στο μηδέν και ο μέσος αριθμός πελατών στην Q_5 αυξάνεται πολύ έντονα. Αυτό οδηγεί και στην απότομη αύξηση του συνολικού end-to-end delay.

Το διάγραμμα επομένως αποτυπώνει καθαρά τον ρόλο της στενωπού: παρότι όλες οι ουρές συμμετέχουν στη συνολική καθυστέρηση, η Q_5 είναι αυτή που καθορίζει τη συμπεριφορά κοντά στο όριο εργοδικότητας.

2.8 Συνολικό συμπέρασμα δεύτερου μέρους

Στο δεύτερο μέρος μελετήσαμε ένα ανοιχτό δίκτυο ουρών αναμονής με πέντε κόμβους. Αρχικά διαβάσαμε τις πιθανότητες δρομολόγησης από το σχήμα και διατυπώσαμε τις εξισώσεις ισορροπίας ροής. Από αυτές προέκυψαν οι συνολικοί ρυθμοί άφιξης:

$$\begin{aligned}\Gamma_1 &= \lambda_1, \\ \Gamma_2 &= \lambda_2 + \frac{1}{2}\lambda_1, \\ \Gamma_3 &= \frac{5}{12}\lambda_1 + \frac{1}{3}\lambda_2, \\ \Gamma_4 &= \frac{1}{3}\lambda_1 + \frac{2}{3}\lambda_2, \\ \Gamma_5 &= \frac{37}{60}\lambda_1 + \frac{11}{15}\lambda_2.\end{aligned}$$

Για τις δοθείσες αριθμητικές τιμές, όλες οι εντάσεις φορτίου ήταν μικρότερες της μονάδας, άρα το δίκτυο είναι εργοδικό. Ο συνολικός μέσος αριθμός πελατών υπολογίστηκε:

$$E[N] = 13.3030108977,$$

και ο μέσος end-to-end χρόνος καθυστέρησης:

$$E[T] = 0.8314381811 \text{ sec.}$$

Η ουρά Q_5 έχει τη μεγαλύτερη ένταση φορτίου:

$$\rho_5 = 0.8805555556,$$

και επομένως αποτελεί τη στενωπό του δικτύου. Η ίδια ουρά καθορίζει και τη μέγιστη τιμή του λ_1 για την οποία το σύστημα παραμένει εργοδικό:

$$\lambda_{1,\max} = 12.3243243243 \text{ πελάτες/sec.}$$

Τέλος, το διάγραμμα του μέσου χρόνου καθυστέρησης ως προς το λ_1 έδειξε την αναμενόμενη συμπεριφορά: η καθυστέρηση αυξάνεται ήπια για μικρές και μεσαίες τιμές του λ_1 , αλλά αυξάνεται απότομα όταν το λ_1 πλησιάζει το μέγιστο επιτρεπτό όριο. Αυτό επιβεβαιώνει τη θεωρητική εικόνα των ουρών $M/M/1$, όπου το μέγεθος $\rho/(1-\rho)$ εκρήγνυται καθώς $\rho \rightarrow 1$.

Συνεπώς, τα αποτελέσματα του Octave και η θεωρητική ανάλυση συμφωνούν πλήρως. Το δίκτυο είναι εργοδικό για τις δοθείσες τιμές, αλλά λειτουργεί σχετικά κοντά στο όριο λόγω της Q_5 , η οποία αποτελεί το κρίσιμο σημείο συμφόρησης.

3 Παράρτημα Υλοποίησης σε Octave

3.1 Κώδικας Υλοποίησης 1ου μέρους

lab5_part1

```
1  function lab5_part1()
2  % =====
3  % Lab 5 - Part A / Theme A
4  % Discrete Event Simulation of a Web Server
5  %
6  % File name:
7  %   lab5_part1.m
8  %
9  % Run in Octave with:
10 %   lab5_part1
11 %
12 % Implements:
13 %   A.1 Single DES run for lambda = 100 req/sec, plots and metrics.
14 %   A.2 Queue/system size plots for lambda in {50,100,150,200,250}.
15 %   A.3 95% confidence intervals from n >= 30 independent replications,
16 %       before and after transient removal.
17 %   A.4 Little's Law verification, with and without transient removal.
18 %
19 % Units:
20 %   - lambda is in requests/sec.
21 %   - service times and simulation time are in milliseconds.
22 %   - inter-arrival mean time is 1000/lambda ms.
23 %
24 % Model:
25 %   - Single FIFO server.
26 %   - Each request first becomes a type-1 job.
27 %   - After type-1 completion, the same request becomes a type-2 job and
28 %     re-enters the back of the same FIFO queue.
29 %   - After type-2 completion, the request leaves the system.
30 % =====
31
32 clc;
33 close all;
34
35 % Try to load the statistics package if available.
36 % The code does not require exprnd/lognrnd because local samplers are used.
37 % tinv/norminv are used if available; otherwise fallback approximations are used.
38 try
39     pkg load statistics;
40 catch
41     printf('Note: statistics package was not loaded. Local samplers/fallback quantiles
42 will be used where needed.\n');
43 end_try_catch
44
45 % Fresh RNG state at the start of the script.
46 rand('state', sum(100 * clock));
47 randn('state', sum(100 * clock) + 12345);
48
49 % =====
50 % User-adjustable simulation parameters
```

```

50 % =====
51
52 max_req_A1      = 1e4;
53 max_req_A2      = 1e4;
54 n_runs_A3       = 30;
55 n_req_rep_A3    = 1e4;
56 warmup_fraction_A3 = 0.30; % MQS-guided conservative transient removal
57 warmup_fraction_A4 = 0.30; % same rule for Little's Law comparison
58
59 % =====
60 % A.1 - Single run for lambda = 100 requests/sec
61 % =====
62
63 printf('\n===== \n');
64 printf('A.1 - Single DES run for lambda = 100 req/sec \n');
65 printf('===== \n');
66
67 lambda_A1 = 100;
68 r = sim_webserver(lambda_A1, max_req_A1);
69
70 R1_A1 = mean(r.resp_t1);
71 R2_A1 = mean(r.resp_t2);
72 R_A1 = mean(r.total_resp);
73
74 tp1_A1 = length(r.resp_t1) / (r.t_end / 1000);
75 tp2_A1 = length(r.resp_t2) / (r.t_end / 1000);
76
77 N1_A1 = tw_mean(r.log_q1, r.log_t);
78 N2_A1 = tw_mean(r.log_q2, r.log_t);
79 N_A1 = tw_mean(r.log_q1 + r.log_q2, r.log_t);
80
81 printf('Simulated %d fully served requests up to t = %.4f ms = %.4f sec \n', ...
82       r.n_served, r.t_end, r.t_end / 1000);
83 printf('Total arrivals observed:      %d \n', r.n_arrived);
84 printf('R1, type-1 phase response:    %.6f ms \n', R1_A1);
85 printf('R2, type-2 phase response:    %.6f ms \n', R2_A1);
86 printf('R end-to-end:                  %.6f ms \n', R_A1);
87 printf('Type-1 jobs served per second: %.6f \n', tp1_A1);
88 printf('Type-2 jobs served per second: %.6f \n', tp2_A1);
89 printf('Mean #type-1 in system, tw:     %.6f \n', N1_A1);
90 printf('Mean #type-2 in system, tw:     %.6f \n', N2_A1);
91 printf('Mean total #jobs in system, tw: %.6f \n', N_A1);
92
93 % A.1 Plot 1: cumulative arrivals and fully served requests.
94 figure(1);
95 clf;
96 plot(r.log_t / 1000, r.log_arrived, 'LineWidth', 1.2);
97 hold on;
98 plot(r.log_t / 1000, r.log_served, 'LineWidth', 1.2);
99 grid on;
100 xlabel('Time (sec)');
101 ylabel('Cumulative requests');
102 title('A.1 - Arrived and fully served requests, lambda = 100 req/sec');
103 legend('Arrived requests', 'Fully served requests', 'Location', 'northwest');

```



```

104
105 % A.1 Plot 2: number of type-1 and type-2 jobs in the system.
106 figure(2);
107 clf;
108
109 subplot(2, 1, 1);
110 plot(r.log_t / 1000, r.log_q1, 'LineWidth', 1.0);
111 grid on;
112 xlabel('Time (sec)');
113 ylabel('Type-1 jobs');
114 title('A.1 - Type-1 jobs in system, queue + service');
115
116 subplot(2, 1, 2);
117 plot(r.log_t / 1000, r.log_q2, 'LineWidth', 1.0);
118 grid on;
119 xlabel('Time (sec)');
120 ylabel('Type-2 jobs');
121 title('A.1 - Type-2 jobs in system, queue + service');
122
123 % A.1 Plot 3: response times and throughput as bar plots.
124 figure(3);
125 clf;
126
127 subplot(1, 2, 1);
128 bar([R1_A1, R2_A1]);
129 grid on;
130 set(gca, 'XTick', 1:2, 'XTickLabel', {'Type 1', 'Type 2'});
131 ylabel('Mean response time (ms)');
132 title('A.1 - Mean response time per phase');
133
134 subplot(1, 2, 2);
135 bar([tp1_A1, tp2_A1]);
136 grid on;
137 set(gca, 'XTick', 1:2, 'XTickLabel', {'Type 1', 'Type 2'});
138 ylabel('Completed jobs per second');
139 title('A.1 - Mean served jobs per second');
140
141 % =====
142 % A.2 - Stationarity across lambda values
143 % =====
144
145 printf('\n===== \n');
146 printf('A.2 - Stationarity across lambda values \n');
147 printf('===== \n');
148
149 S1_mean = exp(1 + 0.5); % E[S1] = exp(mu + sigma^2/2), mu = sigma = 1
150 S2_mean = 0.75; % E[S2] = (0.5 + 1) / 2
151 S_total = S1_mean + S2_mean;
152
153 lambda_stability_bound = 1000 / S_total;
154
155 printf('E[S1] = exp(1.5) = %.6f ms \n', S1_mean);
156 printf('E[S2] = %.6f ms \n', S2_mean);
157 printf('E[S1] + E[S2] = %.6f ms \n', S_total);

```

```

158 printf('Analytical stability condition: rho = lambda*(E[S1]+E[S2])/1000 < 1\n');
159 printf('Stability bound: lambda < %.6f req/sec\n', lambda_stability_bound);
160
161 lambdas_A2 = [50, 100, 150, 200, 250];
162 runs_A2 = cell(1, length(lambdas_A2));
163
164 figure(4);
165 clf;
166
167 for k = 1:length(lambdas_A2)
168     lam = lambdas_A2(k);
169     rho = lam * S_total / 1000;
170
171     if rho < 1
172         status = 'stable';
173     else
174         status = 'unstable';
175     endif
176
177     printf('Running lambda = %d req/sec, rho = %.6f -> %s ... ', lam, rho, status);
178     fflush(stdout);
179
180     runs_A2{k} = sim_webserver(lam, max_req_A2);
181
182     printf('done, t_end = %.4f sec, arrivals = %d, served = %d\n', ...
183         runs_A2{k}.t_end / 1000, runs_A2{k}.n_arrived, runs_A2{k}.n_served);
184
185     q_total = runs_A2{k}.log_q1 + runs_A2{k}.log_q2;
186
187     subplot(length(lambdas_A2), 1, k);
188     plot(runs_A2{k}.log_t / 1000, q_total, 'LineWidth', 1.0);
189     grid on;
190     xlabel('Time (sec)');
191     ylabel('Total jobs');
192     title(sprintf('A.2 - lambda = %d req/sec, rho = %.4f, %s', lam, rho, status));
193 endfor
194
195 printf('\nAnalytical classification for A.2:\n');
196 for k = 1:length(lambdas_A2)
197     lam = lambdas_A2(k);
198     rho = lam * S_total / 1000;
199
200     if rho < 1
201         printf(' lambda = %3d req/sec: rho = %.6f < 1 -> stationary\n', lam, rho);
202     else
203         printf(' lambda = %3d req/sec: rho = %.6f >= 1 -> non-stationary\n', lam,
204             rho);
205     endif
206 endfor
207
208 % =====
209 % A.3 - Confidence intervals before and after transient removal
210 % =====
211

```

```

211 printf('\n===== \n');
212 printf('A.3 - Confidence intervals from independent replications\n');
213 printf('===== \n');
214
215 lambdas_A3 = [100, 180];
216
217 CI1_mean_all = zeros(length(lambdas_A3), 2);
218 CI1_med_all = zeros(length(lambdas_A3), 2);
219 CI2_mean_all = zeros(length(lambdas_A3), 2);
220 CI2_med_all = zeros(length(lambdas_A3), 2);
221
222 CI1_mean_tr = zeros(length(lambdas_A3), 2);
223 CI1_med_tr = zeros(length(lambdas_A3), 2);
224 CI2_mean_tr = zeros(length(lambdas_A3), 2);
225 CI2_med_tr = zeros(length(lambdas_A3), 2);
226
227 % Pilot MQS plots for cutoff visualization.
228 figure(5);
229 clf;
230
231 for li = 1:length(lambdas_A3)
232     lam = lambdas_A3(li);
233
234     pilot = sim_webserver(lam, n_req_rep_A3);
235     q_pilot_total = pilot.log_q1 + pilot.log_q2;
236     mqs_pilot = running_tw_mean(q_pilot_total, pilot.log_t);
237     cutoff_pilot = warmup_fraction_A3 * pilot.t_end;
238
239     subplot(length(lambdas_A3), 1, li);
240     plot(pilot.log_t / 1000, mqs_pilot, 'LineWidth', 1.0);
241     hold on;
242     yl = ylim();
243     plot([cutoff_pilot / 1000, cutoff_pilot / 1000], yl, '--', 'LineWidth', 1.0);
244     grid on;
245     xlabel('Time (sec)');
246     ylabel('MQS(t)');
247     title(sprintf('A.3 - MQS(t), lambda = %d req/sec, cutoff = %.0f%% of run time',
248 ...
249 ... lam, 100 * warmup_fraction_A3));
249     legend('MQS(t)', 'Transient cutoff', 'Location', 'southeast');
250 endfor
251
252 for li = 1:length(lambdas_A3)
253     lam = lambdas_A3(li);
254
255     N1_all = zeros(n_runs_A3, 1);
256     N2_all = zeros(n_runs_A3, 1);
257     N1_tr = zeros(n_runs_A3, 1);
258     N2_tr = zeros(n_runs_A3, 1);
259
260     printf('\nlambda = %d req/sec: running %d independent replications ', ...
261           lam, n_runs_A3);
262
263     for rep = 1:n_runs_A3

```

```

264         rk = sim_webserver(lam, n_req_rep_A3);
265
266         N1_all(rep) = tw_mean(rk.log_q1, rk.log_t);
267         N2_all(rep) = tw_mean(rk.log_q2, rk.log_t);
268
269         cutoff_ms = warmup_fraction_A3 * rk.t_end;
270
271         N1_tr(rep) = tw_mean_interval(rk.log_q1, rk.log_t, cutoff_ms, rk.t_end);
272         N2_tr(rep) = tw_mean_interval(rk.log_q2, rk.log_t, cutoff_ms, rk.t_end);
273
274         printf('.');
275         fflush(stdout);
276     endfor
277
278     printf(' done\n');
279
280     [ci1_mean_all, ci1_med_all] = calc_ci(N1_all, 0.05);
281     [ci2_mean_all, ci2_med_all] = calc_ci(N2_all, 0.05);
282
283     [ci1_mean_tr, ci1_med_tr] = calc_ci(N1_tr, 0.05);
284     [ci2_mean_tr, ci2_med_tr] = calc_ci(N2_tr, 0.05);
285
286     CI1_mean_all(li, :) = ci1_mean_all;
287     CI1_med_all(li, :) = ci1_med_all;
288     CI2_mean_all(li, :) = ci2_mean_all;
289     CI2_med_all(li, :) = ci2_med_all;
290
291     CI1_mean_tr(li, :) = ci1_mean_tr;
292     CI1_med_tr(li, :) = ci1_med_tr;
293     CI2_mean_tr(li, :) = ci2_mean_tr;
294     CI2_med_tr(li, :) = ci2_med_tr;
295
296     printf(' Full trace, type 1: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
297           ci1_mean_all(1), ci1_mean_all(2), ci1_med_all(1), ci1_med_all(2));
298     printf(' Full trace, type 2: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
299           ci2_mean_all(1), ci2_mean_all(2), ci2_med_all(1), ci2_med_all(2));
300
301     printf(' After transient removal, type 1: mean CI [%.6f, %.6f], median CI [%.6f,
302           %.6f]\n', ...
303           ci1_mean_tr(1), ci1_mean_tr(2), ci1_med_tr(1), ci1_med_tr(2));
304     printf(' After transient removal, type 2: mean CI [%.6f, %.6f], median CI [%.6f,
305           %.6f]\n', ...
306           ci2_mean_tr(1), ci2_mean_tr(2), ci2_med_tr(1), ci2_med_tr(2));
307
308     endfor
309
310     % =====
311     % A.4 - Little's Law verification for lambda = 100 requests/sec
312     % =====
313
314     printf('\n=====');
315     printf('A.4 - Little's Law verification for lambda = 100 req/sec\n');
316     printf('=====');
317
318     cutoff_A4 = warmup_fraction_A4 * r.t_end;

```

```

316
317 [N_sim_full, R_full, N_little_full, err_full] = little_stats(r, lambda_A1, 0);
318 [N_sim_tr, R_tr, N_little_tr, err_tr] = little_stats(r, lambda_A1, cutoff_A4);
319
320 printf('Without transient removal:\n');
321 printf('  R = %.6f ms\n', R_full);
322 printf('  N_sim, time-weighted total jobs = %.6f\n', N_sim_full);
323 printf('  lambda * R = (%.6f req/ms) * %.6f ms = %.6f\n', ...
324         lambda_A1 / 1000, R_full, N_little_full);
325 printf('  Percent error = %.6f %%\n', err_full);
326
327 printf('\nWith transient removal:\n');
328 printf('  Cutoff = %.6f ms = %.6f sec, %.0f%% of run time\n', ...
329         cutoff_A4, cutoff_A4 / 1000, 100 * warmup_fraction_A4);
330 printf('  R = %.6f ms\n', R_tr);
331 printf('  N_sim, time-weighted total jobs after cutoff = %.6f\n', N_sim_tr);
332 printf('  lambda * R = (%.6f req/ms) * %.6f ms = %.6f\n', ...
333         lambda_A1 / 1000, R_tr, N_little_tr);
334 printf('  Percent error = %.6f %%\n', err_tr);
335
336 % =====
337 % Final summary
338 % =====
339
340 printf('\n=====');
341 printf('FINAL SUMMARY - Lab 5, Part A\n');
342 printf('=====');
343
344 printf('\nA.1  lambda = %d req/sec, %d fully served requests\n', lambda_A1,
345         max_req_A1);
346 printf('  t_end = %.6f ms = %.6f sec\n', r.t_end, r.t_end / 1000);
347 printf('  arrivals = %d, fully served = %d\n', r.n_arrived, r.n_served);
348 printf('  R1 = %.6f ms, R2 = %.6f ms, R = %.6f ms\n', R1_A1, R2_A1, R_A1);
349 printf('  throughput: type-1 = %.6f jobs/sec, type-2 = %.6f jobs/sec\n', tp1_A1,
350         tp2_A1);
351 printf('  time-weighted mean jobs: N1 = %.6f, N2 = %.6f, N = %.6f\n', ...
352         N1_A1, N2_A1, N_A1);
353
354 printf('\nA.2  Stability bound\n');
355 printf('  lambda < %.6f req/sec\n', lambda_stability_bound);
356
357 for k = 1:length(lambdas_A2)
358     lam = lambdas_A2(k);
359     rho = lam * S_total / 1000;
360
361     if rho < 1
362         printf('  lambda = %3d: rho = %.6f -> stationary\n', lam, rho);
363     else
364         printf('  lambda = %3d: rho = %.6f -> non-stationary\n', lam, rho);
365     endif
366 endfor
367
368 printf('\nA.3  95%% confidence intervals, n = %d replications\n', n_runs_A3);

```

```

367 printf(' Transient removal rule: remove first %.0f%% of each run, guided by MQS(t)
plots.\n', ...
368         100 * warmup_fraction_A3);
369
370 for li = 1:length(lambdas_A3)
371     lam = lambdas_A3(li);
372
373     printf('\n lambda = %d req/sec\n', lam);
374
375     printf(' Full trace:\n');
376     printf(' type 1: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
377         CI1_mean_all(li, 1), CI1_mean_all(li, 2), ...
378         CI1_med_all(li, 1), CI1_med_all(li, 2));
379     printf(' type 2: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
380         CI2_mean_all(li, 1), CI2_mean_all(li, 2), ...
381         CI2_med_all(li, 1), CI2_med_all(li, 2));
382
383     printf(' After transient removal:\n');
384     printf(' type 1: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
385         CI1_mean_tr(li, 1), CI1_mean_tr(li, 2), ...
386         CI1_med_tr(li, 1), CI1_med_tr(li, 2));
387     printf(' type 2: mean CI [%.6f, %.6f], median CI [%.6f, %.6f]\n', ...
388         CI2_mean_tr(li, 1), CI2_mean_tr(li, 2), ...
389         CI2_med_tr(li, 1), CI2_med_tr(li, 2));
390 endfor
391
392 printf('\nA.4 Little''s Law, lambda = %d req/sec\n', lambda_A1);
393 printf(' Without transient removal:\n');
394 printf(' R = %.6f ms, N_sim = %.6f, lambda*R = %.6f, error = %.6f %%\n', ...
395         R_full, N_sim_full, N_little_full, err_full);
396 printf(' With transient removal:\n');
397 printf(' R = %.6f ms, N_sim = %.6f, lambda*R = %.6f, error = %.6f %%\n', ...
398         R_tr, N_sim_tr, N_little_tr, err_tr);
399
400 printf('\nDone. ');
401
402 endfunction
403
404 % =====
405 % Local functions
406 % =====
407
408 function res = sim_webserver(lambda, max_req)
409     % Run one DES simulation of the web server.
410     %
411     % lambda : external request arrival rate in requests/sec
412     % max_req : stop after this many requests have fully finished
413     %
414     % Returned fields:
415     % log_t, log_q1, log_q2, log_arrived, log_served
416     % resp_t1, resp_t2, total_resp
417     % completed_depart2, completed_arrival
418     % n_arrived, n_served, t_end
419

```

```

420     ARRIVAL = 1;
421     DEPART1 = 2;
422     DEPART2 = 3;
423
424     inter_arr_mean = 1000 / lambda;           % ms
425
426     % Event list rows: [time, event_type]
427     events = [sample_exp_mean(inter_arr_mean), ARRIVAL];
428
429     % FIFO queue rows: [job_type, phase_start_time, request_id]
430     queue = [];
431
432     busy = false;
433     cur_job = [0, 0, 0];                      % [job_type, phase_start_time, request_id]
434
435     % Logs start at t = 0 with an empty system.
436     log_t      = 0;
437     log_q1     = 0;
438     log_q2     = 0;
439     log_arrived = 0;
440     log_served  = 0;
441
442     resp_t1 = [];
443     resp_t2 = [];
444     total_resp = [];
445
446     arrival_times = [];
447     depart1_times = [];
448
449     completed_ids = [];
450     completed_arrival = [];
451     completed_depart2 = [];
452
453     n_arrived = 0;
454     n_served  = 0;
455     t = 0;
456
457     while n_served < max_req
458         if isempty(events)
459             break;
460         endif
461
462         % Pick the event with the smallest timestamp.
463         [t, idx] = min(events(:, 1));
464         etype = events(idx, 2);
465         events(idx, :) = [];
466
467         if etype == ARRIVAL
468             n_arrived = n_arrived + 1;
469             req_id = n_arrived;
470
471             arrival_times(req_id) = t;
472
473             % Schedule the next external arrival.

```

```

474         events = [events; t + sample_exp_mean(inter_arr_mean), ARRIVAL];
475
476         if ~busy
477             busy = true;
478             cur_job = [1, t, req_id];
479             events = [events; t + sample_lognormal(1, 1), DEPART1];
480         else
481             queue = [queue; 1, t, req_id];
482         endif
483
484     elseif etype == DEPART1
485         req_id = cur_job(3);
486
487         depart1_times(req_id) = t;
488         resp_t1 = [resp_t1, t - cur_job(2)];
489
490         % The same request becomes a type-2 job at the back of the FIFO queue.
491         queue = [queue; 2, t, req_id];
492
493         % Start the next job from the FIFO queue.
494         [cur_job, queue] = pop_next(queue);
495
496         if cur_job(1) == 1
497             events = [events; t + sample_lognormal(1, 1), DEPART1];
498         else
499             events = [events; t + sample_uniform(0.5, 1.0), DEPART2];
500         endif
501
502     elseif etype == DEPART2
503         req_id = cur_job(3);
504
505         resp_t2 = [resp_t2, t - cur_job(2)];
506         total_resp = [total_resp, t - arrival_times(req_id)];
507
508         completed_ids = [completed_ids, req_id];
509         completed_arrival = [completed_arrival, arrival_times(req_id)];
510         completed_depart2 = [completed_depart2, t];
511
512         n_served = n_served + 1;
513
514         if ~isempty(queue)
515             [cur_job, queue] = pop_next(queue);
516
517             if cur_job(1) == 1
518                 events = [events; t + sample_lognormal(1, 1), DEPART1];
519             else
520                 events = [events; t + sample_uniform(0.5, 1.0), DEPART2];
521             endif
522         else
523             busy = false;
524             cur_job = [0, 0, 0];
525         endif
526     endif
527

```



```

528         % Count type-1 and type-2 jobs in the system: FIFO queue + current service.
529         if isempty(queue)
530             nq1 = 0;
531             nq2 = 0;
532         else
533             nq1 = sum(queue(:, 1) == 1);
534             nq2 = sum(queue(:, 1) == 2);
535         endif
536
537         nq1 = nq1 + (cur_job(1) == 1);
538         nq2 = nq2 + (cur_job(1) == 2);
539
540         log_t      = [log_t,      t];
541         log_q1     = [log_q1,     nq1];
542         log_q2     = [log_q2,     nq2];
543         log_arrived = [log_arrived, n_arrived];
544         log_served  = [log_served,  n_served];
545     endwhile
546
547     res.log_t      = log_t;
548     res.log_q1     = log_q1;
549     res.log_q2     = log_q2;
550     res.log_arrived = log_arrived;
551     res.log_served  = log_served;
552
553     res.resp_t1 = resp_t1;
554     res.resp_t2 = resp_t2;
555     res.total_resp = total_resp;
556
557     res.completed_ids = completed_ids;
558     res.completed_arrival = completed_arrival;
559     res.completed_depart2 = completed_depart2;
560
561     res.n_arrived = n_arrived;
562     res.n_served  = n_served;
563     res.t_end = t;
564 endfunction
565
566 function [job, queue_out] = pop_next(queue)
567     % FIFO pop.
568     job = queue(1, :);
569     queue_out = queue(2:end, :);
570 endfunction
571
572 function x = sample_exp_mean(mean_value)
573     % Exponential random variable with the given mean.
574     u = rand();
575
576     while u <= 0
577         u = rand();
578     endwhile
579
580     x = -mean_value * log(u);
581 endfunction

```

```

582
583 function x = sample_lognormal(mu, sigma)
584     % LogNormal(mu, sigma), where mu/sigma are the parameters of the
585     % underlying normal distribution.
586     x = exp(mu + sigma * randn());
587 endfunction
588
589 function x = sample_uniform(a, b)
590     % Uniform(a,b).
591     x = a + (b - a) * rand();
592 endfunction
593
594 function avg = tw_mean(vals, times)
595     % Time-weighted average of a step function.
596     % vals(i) is assumed to hold over [times(i), times(i+1)).
597     avg = tw_mean_interval(vals, times, times(1), times(end));
598 endfunction
599
600 function avg = tw_mean_interval(vals, times, t_start, t_stop)
601     % Time-weighted average of a step function over [t_start, t_stop].
602     vals = vals(:)';
603     times = times(:)';
604
605     if length(times) < 2
606         avg = NaN;
607         return;
608     endif
609
610     t_start = max(t_start, times(1));
611     t_stop = min(t_stop, times(end));
612
613     if t_stop <= t_start
614         avg = NaN;
615         return;
616     endif
617
618     area = 0;
619     total_time = 0;
620
621     for i = 1:(length(times) - 1)
622         a = max(times(i), t_start);
623         b = min(times(i + 1), t_stop);
624
625         if b > a
626             area = area + vals(i) * (b - a);
627             total_time = total_time + (b - a);
628         endif
629     endfor
630
631     if total_time <= 0
632         avg = NaN;
633     else
634         avg = area / total_time;
635     endif

```

```

636 endfunction
637
638 function mqs = running_tw_mean(vals, times)
639     % Running time-weighted average:
640     %  $MQS(t_j) = (1 / (t_j - t_0)) * \int_{t_0}^{t_j} Q(s) ds$ .
641     vals = vals(:)';
642     times = times(:)';
643
644     n = length(times);
645     mqs = zeros(1, n);
646
647     if n < 2
648         return;
649     endif
650
651     area = 0;
652     t0 = times(1);
653     mqs(1) = vals(1);
654
655     for i = 1:(n - 1)
656         dt = times(i + 1) - times(i);
657
658         if dt < 0
659             error('Times must be non-decreasing.');
```

```

690
691     n = length(xs);
692
693     if n < 2
694         ci_mean = [NaN, NaN];
695         ci_med  = [NaN, NaN];
696         return;
697     endif
698
699     xbar = mean(xs);
700     s = std(xs);
701
702     eta = t_quantile(1 - alpha / 2, n - 1);
703     ci_mean = [xbar - eta * s / sqrt(n), xbar + eta * s / sqrt(n)];
704
705     z = normal_quantile(1 - alpha / 2);
706     k = floor(n / 2 - z * sqrt(n) / 2);
707     k = max(k, 1);
708     k = min(k, floor((n + 1) / 2));
709
710     ci_med = [xs(k), xs(n + 1 - k)];
711 endfunction
712
713 function q = normal_quantile(p)
714     % Normal quantile. Uses norminv if available, otherwise erfinv formula.
715     if exist('norminv', 'file') || exist('norminv', 'builtin')
716         q = norminv(p);
717     else
718         q = sqrt(2) * erfinv(2 * p - 1);
719     endif
720 endfunction
721
722 function q = t_quantile(p, df)
723     % Student-t quantile.
724     % Uses tinv if available. If not available, uses standard normal as a
725     % fallback, except for df = 29 where the common n = 30 value is included.
726     if exist('tinv', 'file') || exist('tinv', 'builtin')
727         q = tinv(p, df);
728     else
729         if df == 29 && abs(p - 0.975) < 1e-12
730             q = 2.045230;
731         else
732             q = normal_quantile(p);
733         endif
734     endif
735 endfunction
736
737 function [N_sim, R_mean, N_little, err_percent] = little_stats(r, lambda, cutoff_ms)
738     % Little's Law verification for one trace.
739     %
740     % N_sim:
741     %     time-weighted mean total number of jobs in the system.
742     %
743     % R_mean:

```

```

744 % mean end-to-end response time of requests completed after cutoff.
745 %
746 % N_little:
747 % lambda_ms * R_mean.
748 %
749 % err_percent:
750 % relative absolute error between N_little and N_sim.
751
752 q_total = r.log_q1 + r.log_q2;
753
754 if cutoff_ms <= 0
755     N_sim = tw_mean(q_total, r.log_t);
756     idx = true(size(r.total_resp));
757 else
758     N_sim = tw_mean_interval(q_total, r.log_t, cutoff_ms, r.t_end);
759     idx = (r.completed_arrival >= cutoff_ms) & (r.completed_depart2 >= cutoff_ms);
760 endif
761
762 if sum(idx) == 0
763     R_mean = NaN;
764     N_little = NaN;
765     err_percent = NaN;
766     return;
767 endif
768
769 R_mean = mean(r.total_resp(idx));
770
771 lambda_ms = lambda / 1000;
772 N_little = lambda_ms * R_mean;
773
774 if abs(N_sim) < eps
775     err_percent = NaN;
776 else
777     err_percent = 100 * abs(N_little - N_sim) / abs(N_sim);
778 endif
779 endfunction
780

```

3.2 Έξοδος Προγράμματος 1ου μέρους

```

=====
A.1 - Single DES run for lambda = 100 req/sec
=====
Simulated 10000 fully served requests up to t = 100837.6373 ms = 100.8376 sec
Total arrivals observed:      10002
R1, type-1 phase response:    10.579663 ms
R2, type-2 phase response:    6.398254 ms
R end-to-end:                 16.977038 ms
Type-1 jobs served per second: 99.179238
Type-2 jobs served per second: 99.169321
Mean #type-1 in system, tw:   1.049398
Mean #type-2 in system, tw:   0.634528
Mean total #jobs in system, tw: 1.683926

=====
A.2 - Stationarity across lambda values
=====
E[S1] = exp(1.5) = 4.481689 ms
E[S2] = 0.750000 ms
E[S1] + E[S2] = 5.231689 ms
Analytical stability condition: rho = lambda*(E[S1]+E[S2])/1000 < 1
Stability bound: lambda < 191.142858 req/sec
Running lambda = 50 req/sec, rho = 0.261584 -> stable ... done, t_end = 203.2197
sec, arrivals =
  10000, served = 10000
Running lambda = 100 req/sec, rho = 0.523169 -> stable ... done, t_end =
100.0762 sec, arrivals
= 10000, served = 10000
Running lambda = 150 req/sec, rho = 0.784753 -> stable ... done, t_end = 67.1597
sec, arrivals =
  10002, served = 10000
Running lambda = 200 req/sec, rho = 1.046338 -> unstable ... done, t_end =
52.5433 sec, arrivals
= 10377, served = 10000
Running lambda = 250 req/sec, rho = 1.307922 -> unstable ... done, t_end =
65.0984 sec, arrivals
= 16152, served = 10000

Analytical classification for A.2:
  lambda = 50 req/sec: rho = 0.261584 < 1 -> stationary
  lambda = 100 req/sec: rho = 0.523169 < 1 -> stationary
  lambda = 150 req/sec: rho = 0.784753 < 1 -> stationary
  lambda = 200 req/sec: rho = 1.046338 >= 1 -> non-stationary
  lambda = 250 req/sec: rho = 1.307922 >= 1 -> non-stationary

=====
A.3 - Confidence intervals from independent replications
=====

lambda = 100 req/sec: running 30 independent
replications ..... done
  Full trace, type 1: mean CI [1.014991, 1.056171], median CI [1.011125,
1.063644]
  Full trace, type 2: mean CI [0.603187, 0.632896], median CI [0.601051,
0.631790]
  After transient removal, type 1: mean CI [1.008903, 1.060815], median CI
[0.984703, 1.060240]
  After transient removal, type 2: mean CI [0.599920, 0.636938], median CI
[0.586471, 0.647216]

lambda = 180 req/sec: running 30 independent
replications ..... done
  Full trace, type 1: mean CI [14.476681, 18.489433], median CI [13.566229,
18.068241]

```

Full trace, type 2: mean CI [13.679712, 17.688760], median CI [12.826022, 17.198553]
 After transient removal, type 1: mean CI [14.638395, 19.446088], median CI [13.488622, 18.944785]
 After transient removal, type 2: mean CI [13.891899, 18.707440], median CI [12.648005, 18.337582]

=====
 A.4 - Little's Law verification for $\lambda = 100$ req/sec
 =====

Without transient removal:

R = 16.977038 ms
 N_sim, time-weighted total jobs = 1.683926
 $\lambda * R = (0.100000 \text{ req/ms}) * 16.977038 \text{ ms} = 1.697704$
 Percent error = 0.818167 %

With transient removal:

Cutoff = 30251.291177 ms = 30.251291 sec, 30% of run time
 R = 16.830559 ms
 N_sim, time-weighted total jobs after cutoff = 1.651705
 $\lambda * R = (0.100000 \text{ req/ms}) * 16.830559 \text{ ms} = 1.683056$
 Percent error = 1.898066 %

=====
 FINAL SUMMARY - Lab 5, Part A
 =====

A.1 $\lambda = 100$ req/sec, 10000 fully served requests
 t_end = 100837.637255 ms = 100.837637 sec
 arrivals = 10002, fully served = 10000
 R1 = 10.579663 ms, R2 = 6.398254 ms, R = 16.977038 ms
 throughput: type-1 = 99.179238 jobs/sec, type-2 = 99.169321 jobs/sec
 time-weighted mean jobs: N1 = 1.049398, N2 = 0.634528, N = 1.683926

A.2 Stability bound

$\lambda < 191.142858$ req/sec
 $\lambda = 50$: $\rho = 0.261584$ -> stationary
 $\lambda = 100$: $\rho = 0.523169$ -> stationary
 $\lambda = 150$: $\rho = 0.784753$ -> stationary
 $\lambda = 200$: $\rho = 1.046338$ -> non-stationary
 $\lambda = 250$: $\rho = 1.307922$ -> non-stationary

A.3 95% confidence intervals, n = 30 replications

Transient removal rule: remove first 30% of each run, guided by MQS(t) plots.

$\lambda = 100$ req/sec

Full trace:

type 1: mean CI [1.014991, 1.056171], median CI [1.011125, 1.063644]
 type 2: mean CI [0.603187, 0.632896], median CI [0.601051, 0.631790]

After transient removal:

type 1: mean CI [1.008903, 1.060815], median CI [0.984703, 1.060240]
 type 2: mean CI [0.599920, 0.636938], median CI [0.586471, 0.647216]

$\lambda = 180$ req/sec

Full trace:

type 1: mean CI [14.476681, 18.489433], median CI [13.566229, 18.068241]
 type 2: mean CI [13.679712, 17.688760], median CI [12.826022, 17.198553]

After transient removal:

type 1: mean CI [14.638395, 19.446088], median CI [13.488622, 18.944785]
 type 2: mean CI [13.891899, 18.707440], median CI [12.648005, 18.337582]

A.4 Little's Law, $\lambda = 100$ req/sec

Without transient removal:

R = 16.977038 ms, N_sim = 1.683926, λR = 1.697704, error = 0.818167 %

With transient removal:

R = 16.830559 ms, N_sim = 1.651705, λR = 1.683056, error = 1.898066 %

Done.

3.3 Κώδικας Υλοποίησης 2ου μέρους

lab5_part2

```
1  function lab5_part2()
2  % =====
3  % Lab 5 - Part B
4  % Open Jackson Queueing Network
5  %
6  % File name:
7  %   lab5_part2.m
8  %
9  % Run in Octave with:
10 %   lab5_part2
11 %
12 % Implements:
13 %   B.2 intensities(lambda1, lambda2, mu1, ..., mu5)
14 %   B.3 mean_clients(lambda1, lambda2, mu1, ..., mu5)
15 %   B.4 numerical evaluation for the given parameters
16 %   B.5 bottleneck queue and maximum lambda1 for ergodicity
17 %   B.6 end-to-end mean delay plot versus lambda1
18 %
19 % Network routing used:
20 %   Q1 -> Q2      with probability 1/2
21 %   Q1 -> Q3      with probability 1/4
22 %   Q1 -> exit    with probability 1/4
23 %
24 %   Q2 -> Q3      with probability 1/3
25 %   Q2 -> Q4      with probability 2/3
26 %
27 %   Q3 -> Q5      with probability 1
28 %
29 %   Q4 -> Q5      with probability 3/5
30 %   Q4 -> exit    with probability 2/5
31 %
32 %   Q5 -> exit    with probability 1
33 %
34 % Effective arrival rates:
35 %   Gamma1 = lambda1
36 %   Gamma2 = lambda2 + (1/2)lambda1
37 %   Gamma3 = (5/12)lambda1 + (1/3)lambda2
38 %   Gamma4 = (1/3)lambda1 + (2/3)lambda2
39 %   Gamma5 = (37/60)lambda1 + (11/15)lambda2
40 %
41 % All rates are in clients/sec, so delays are in seconds.
42 % =====
43
44 clc;
45 close all;
46 format long g;
47
48 % -----
49 % Given numerical parameters from the statement
50 % -----
```

```

51
52 lambda1 = 10;
53 lambda2 = 6;
54
55 mu1 = 15;
56 mu2 = 20;
57 mu3 = 10;
58 mu4 = 14;
59 mu5 = 12;
60
61 mu = [mu1, mu2, mu3, mu4, mu5];
62
63 fprintf('\n===== \n');
64 fprintf('Lab 5 - Part B: Open Jackson Queueing Network \n');
65 fprintf('===== \n');
66
67 fprintf('\nInput parameters: \n');
68 fprintf(' lambda1 = %.10f clients/sec \n', lambda1);
69 fprintf(' lambda2 = %.10f clients/sec \n', lambda2);
70 fprintf(' mu1 = %.10f clients/sec \n', mu1);
71 fprintf(' mu2 = %.10f clients/sec \n', mu2);
72 fprintf(' mu3 = %.10f clients/sec \n', mu3);
73 fprintf(' mu4 = %.10f clients/sec \n', mu4);
74 fprintf(' mu5 = %.10f clients/sec \n', mu5);
75
76 % -----
77 % B.2 - Effective arrival rates and intensities
78 % -----
79
80 fprintf('\n===== \n');
81 fprintf('B.2 - Effective arrival rates and traffic intensities \n');
82 fprintf('===== \n');
83
84 gamma = effective_arrival_rates(lambda1, lambda2);
85
86 fprintf('\nEffective arrival rates Gamma_i: \n');
87 for i = 1:5
88     fprintf(' Gamma%d = %.10f clients/sec \n', i, gamma(i));
89 endfor
90
91 [rho, ergodic] = intensities(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5);
92
93 fprintf('\nErgodicity flag: %d \n', ergodic);
94 if ergodic == 1
95     fprintf('The network is ergodic because all rho_i are smaller than 1. \n');
96 else
97     fprintf('The network is not ergodic because at least one rho_i is >= 1. \n');
98 endif
99
100 % -----
101 % B.3 - Mean number of clients in each queue
102 % -----
103
104 fprintf('\n===== \n');

```

```

105 fprintf('B.3 - Mean number of clients in each queue\n');
106 fprintf('=====\n');
107
108 N = mean_clients(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5);
109
110 fprintf('\nMean clients E[N_i] in each queue:\n');
111 for i = 1:5
112     fprintf(' E[N%d] = %.10f\n', i, N(i));
113 endfor
114
115 N_total = sum(N);
116
117 fprintf('\nTotal mean number of clients in the network:\n');
118 fprintf(' E[N] = sum_i E[N_i] = %.10f\n', N_total);
119
120 % -----
121 % B.4 - End-to-end mean delay for the given parameters
122 % -----
123
124 fprintf('\n=====\n');
125 fprintf('B.4 - End-to-end mean delay for the given parameters\n');
126 fprintf('=====\n');
127
128 external_rate = lambda1 + lambda2;
129 T_end_to_end = end_to_end_delay(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5);
130
131 fprintf('\nTotal external arrival rate:\n');
132 fprintf(' gamma_external = lambda1 + lambda2 = %.10f clients/sec\n', external_rate);
133
134 fprintf('\nEnd-to-end mean delay by Little''s Law:\n');
135 fprintf(' E[T] = E[N] / gamma_external = %.10f seconds\n', T_end_to_end);
136
137 % -----
138 % B.5 - Bottleneck and maximum lambda1 for ergodicity
139 % -----
140
141 fprintf('\n=====\n');
142 fprintf('B.5 - Bottleneck queue and maximum lambda1\n');
143 fprintf('=====\n');
144
145 [max_rho, bottleneck_current] = max(rho);
146
147 fprintf('\nCurrent bottleneck queue, based on maximum rho_i:\n');
148 fprintf(' bottleneck = Q%d\n', bottleneck_current);
149 fprintf(' max rho = %.10f\n', max_rho);
150
151 [lambda1_max, limiting_queue, bounds] = lambda1_max_for_ergodicity(lambda2, mu1, mu2,
mu3, mu4, mu5);
152
153 fprintf('\nUpper bounds on lambda1 imposed by each queue:\n');
154 for i = 1:5
155     fprintf(' From Q%d: lambda1 < %.10f\n', i, bounds(i));
156 endfor
157

```

```

158 fprintf('\nMaximum lambda1 for ergodicity with lambda2 fixed at %.10f:\n', lambda2);
159 fprintf(' lambda1_max = %.10f clients/sec\n', lambda1_max);
160 fprintf(' limiting queue = Q%d\n', limiting_queue);
161
162 if lambda1 < lambda1_max
163     fprintf(' Current lambda1 = %.10f is below lambda1_max, so the current network is
ergodic.\n', lambda1);
164 else
165     fprintf(' Current lambda1 = %.10f is not below lambda1_max, so the current
network is not ergodic.\n', lambda1);
166 endif
167
168 % -----
169 % B.6 - Plot end-to-end delay versus lambda1
170 % -----
171
172 fprintf('\n===== \n');
173 fprintf('B.6 - End-to-end delay plot versus lambda1\n');
174 fprintf('===== \n');
175
176 lambda1_values = linspace(0.1 * lambda1_max, 0.99 * lambda1_max, 300);
177 delay_values = zeros(size(lambda1_values));
178
179 for k = 1:length(lambda1_values)
180     delay_values(k) = end_to_end_delay(lambda1_values(k), lambda2, ...
181                                     mu1, mu2, mu3, mu4, mu5);
182 endfor
183
184 fprintf('\nPlot range:\n');
185 fprintf(' lambda1_min = %.10f clients/sec\n', lambda1_values(1));
186 fprintf(' lambda1_max_plot = %.10f clients/sec\n', lambda1_values(end));
187 fprintf(' lambda1_max_ergodic = %.10f clients/sec\n', lambda1_max);
188
189 fprintf('\nDelay values at selected points:\n');
190
191 selected_factors = [0.10, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99];
192 for k = 1:length(selected_factors)
193     l1_sel = selected_factors(k) * lambda1_max;
194     d_sel = end_to_end_delay(l1_sel, lambda2, mu1, mu2, mu3, mu4, mu5);
195
196     fprintf(' lambda1 = %.2f * lambda1_max = %.10f -> E[T] = %.10f sec\n', ...
197           selected_factors(k), l1_sel, d_sel);
198 endfor
199
200 figure(1);
201 clf;
202
203 plot(lambda1_values, delay_values, 'LineWidth', 1.5);
204 grid on;
205 xlabel('lambda_1 (clients/sec)');
206 ylabel('End-to-end mean delay E[T] (sec)');
207 title('B.6 - End-to-end mean delay versus lambda_1');
208
209 hold on;

```

```

210     yl = ylim();
211     plot([lambda1, lambda1], yl, '--', 'LineWidth', 1.0);
212     plot([lambda1_max, lambda1_max], yl, ':', 'LineWidth', 1.0);
213     legend('E[T]', 'Given lambda_1 = 10', 'lambda_{1,max}', 'Location', 'northwest');
214     hold off;
215
216     try
217         print(gcf, 'lab5_part2_delay.png', '-dpng', '-r200');
218         fprintf('\nSaved figure as lab5_part2_delay.png\n');
219     catch
220         fprintf('\nCould not save figure automatically. Please export it manually if
needed.\n');
221     end_try_catch
222
223     % -----
224     % Final summary for easy copy into report
225     % -----
226
227     fprintf('\n===== \n');
228     fprintf('FINAL SUMMARY - Lab 5, Part B \n');
229     fprintf('===== \n');
230
231     fprintf('\nRouting equations: \n');
232     fprintf('    Gamma1 = lambda1 \n');
233     fprintf('    Gamma2 = lambda2 + (1/2)lambda1 \n');
234     fprintf('    Gamma3 = (5/12)lambda1 + (1/3)lambda2 \n');
235     fprintf('    Gamma4 = (1/3)lambda1 + (2/3)lambda2 \n');
236     fprintf('    Gamma5 = (37/60)lambda1 + (11/15)lambda2 \n');
237
238     fprintf('\nGiven parameters: \n');
239     fprintf('    lambda1 = %.10f, lambda2 = %.10f \n', lambda1, lambda2);
240     fprintf('    mu = [%.10f %.10f %.10f %.10f %.10f] \n', mu1, mu2, mu3, mu4, mu5);
241
242     fprintf('\nEffective arrival rates: \n');
243     fprintf('    Gamma = [ ');
244     fprintf(' %.10f', gamma);
245     fprintf(' ] \n');
246
247     fprintf('\nTraffic intensities: \n');
248     fprintf('    rho = [ ');
249     fprintf(' %.10f', rho);
250     fprintf(' ] \n');
251     fprintf('    ergodic = %d \n', ergodic);
252
253     fprintf('\nMean clients: \n');
254     fprintf('    E[N_i] = [ ');
255     fprintf(' %.10f', N);
256     fprintf(' ] \n');
257     fprintf('    E[N] = %.10f \n', N_total);
258
259     fprintf('\nEnd-to-end delay: \n');
260     fprintf('    E[T] = %.10f seconds \n', T_end_to_end);
261
262     fprintf('\nBottleneck: \n');

```

```

263 fprintf(' current bottleneck = %d, rho = %.10f\n', bottleneck_current, max_rho);
264 fprintf(' lambda1_max = %.10f clients/sec\n', lambda1_max);
265 fprintf(' limiting queue for lambda1_max = %d\n', limiting_queue);
266
267 fprintf('\nDone.');
```

```

268
269 endfunction
270
271 % =====
272 % Required function for B.2
273 % =====
274
275 function [rho, ergodic] = intensities(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5,
verbose)
276     % Computes traffic intensities rho_i for the five queues.
277     %
278     % Required public use:
279     % [rho, ergodic] = intensities(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5)
280     %
281     % Optional internal use:
282     % verbose = false suppresses printing.
283
284     if nargin < 8
285         verbose = true;
286     endif
287
288     mu = [mu1, mu2, mu3, mu4, mu5];
289     gamma = effective_arrival_rates(lambda1, lambda2);
290
291     rho = gamma ./ mu;
292     ergodic = all(rho < 1);
293
294     if verbose
295         fprintf('\nTraffic intensities rho_i = Gamma_i / mu_i:\n');
296         for i = 1:5
297             fprintf(' rho%d = %.10f\n', i, rho(i));
298         endfor
299     endif
300 endfunction
301
302 % =====
303 % Required function for B.3
304 % =====
305
306 function N = mean_clients(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5)
307     % Computes the mean number of clients in each M/M/1 queue:
308     %  $E[N_i] = \rho_i / (1 - \rho_i)$ 
309     %
310     % If the system is not ergodic, returns NaN values.
311
312     [rho, ergodic] = intensities(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5, false);
313
314     if ergodic == 0
315         N = NaN(1, 5);

```



```

316         return;
317     endif
318
319     N = rho ./ (1 - rho);
320 endfunction
321
322 % =====
323 % Helper functions
324 % =====
325
326 function gamma = effective_arrival_rates(lambda1, lambda2)
327     % Effective arrival rates at each queue, derived from the routing graph.
328
329     gamma1 = lambda1;
330     gamma2 = lambda2 + (1/2) * lambda1;
331     gamma3 = (5/12) * lambda1 + (1/3) * lambda2;
332     gamma4 = (1/3) * lambda1 + (2/3) * lambda2;
333     gamma5 = (37/60) * lambda1 + (11/15) * lambda2;
334
335     gamma = [gamma1, gamma2, gamma3, gamma4, gamma5];
336 endfunction
337
338 function T = end_to_end_delay(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5)
339     % Computes end-to-end mean delay using Little's Law:
340     %  $E[T] = E[N] / (\lambda_1 + \lambda_2)$ 
341
342     N = mean_clients(lambda1, lambda2, mu1, mu2, mu3, mu4, mu5);
343     external_rate = lambda1 + lambda2;
344
345     if any(isnan(N)) || external_rate <= 0
346         T = NaN;
347         return;
348     endif
349
350     T = sum(N) / external_rate;
351 endfunction
352
353 function [lambda1_max, limiting_queue, bounds] = lambda1_max_for_ergodicity(lambda2,
mu1, mu2, mu3, mu4, mu5)
354     % Computes the maximum lambda1 such that all queues remain ergodic,
355     % with lambda2 and all mu_i fixed.
356     %
357     % Conditions:
358     % Gamma1 < mu1
359     % Gamma2 < mu2
360     % Gamma3 < mu3
361     % Gamma4 < mu4
362     % Gamma5 < mu5
363     %
364     % where:
365     % Gamma1 = lambda1
366     % Gamma2 = lambda2 + (1/2)lambda1
367     % Gamma3 = (5/12)lambda1 + (1/3)lambda2
368     % Gamma4 = (1/3)lambda1 + (2/3)lambda2

```

```
369 % Gamma5 = (37/60)lambda1 + (11/15)lambda2
370
371 bound1 = mu1;
372 bound2 = 2 * (mu2 - lambda2);
373 bound3 = (12/5) * (mu3 - (1/3) * lambda2);
374 bound4 = 3 * (mu4 - (2/3) * lambda2);
375 bound5 = (60/37) * (mu5 - (11/15) * lambda2);
376
377 bounds = [bound1, bound2, bound3, bound4, bound5];
378
379 [lambda1_max, limiting_queue] = min(bounds);
380
381 if lambda1_max < 0
382     lambda1_max = NaN;
383 endif
384 endfunction
385
```

3.4 Έξοδος Προγράμματος 2ου μέρους

=====

Lab 5 - Part B: Open Jackson Queueing Network

=====

Input parameters:

lambda1 = 10.0000000000 clients/sec
lambda2 = 6.0000000000 clients/sec
mu1 = 15.0000000000 clients/sec
mu2 = 20.0000000000 clients/sec
mu3 = 10.0000000000 clients/sec
mu4 = 14.0000000000 clients/sec
mu5 = 12.0000000000 clients/sec

=====

B.2 - Effective arrival rates and traffic intensities

=====

Effective arrival rates Γ_i :

Gamma1 = 10.0000000000 clients/sec
Gamma2 = 11.0000000000 clients/sec
Gamma3 = 6.1666666667 clients/sec
Gamma4 = 7.3333333333 clients/sec
Gamma5 = 10.5666666667 clients/sec

Traffic intensities $\rho_i = \Gamma_i / \mu_i$:

rho1 = 0.6666666667
rho2 = 0.5500000000
rho3 = 0.6166666667
rho4 = 0.5238095238
rho5 = 0.8805555556

Ergodicity flag: 1

The network is ergodic because all ρ_i are smaller than 1.

=====

B.3 - Mean number of clients in each queue

=====

Mean clients $E[N_i]$ in each queue:

E[N1] = 2.0000000000
E[N2] = 1.2222222222
E[N3] = 1.6086956522
E[N4] = 1.1000000000
E[N5] = 7.3720930233

Total mean number of clients in the network:

$E[N] = \sum_i E[N_i] = 13.3030108977$

=====

B.4 - End-to-end mean delay for the given parameters

=====

Total external arrival rate:

gamma_external = lambda1 + lambda2 = 16.0000000000 clients/sec

End-to-end mean delay by Little's Law:

$E[T] = E[N] / \text{gamma_external} = 0.8314381811$ seconds

=====

B.5 - Bottleneck queue and maximum lambda1

=====

Current bottleneck queue, based on maximum ρ_i :

bottleneck = Q5

max rho = 0.8805555556

Upper bounds on lambda1 imposed by each queue:

From Q1: lambda1 < 15.0000000000
From Q2: lambda1 < 28.0000000000
From Q3: lambda1 < 19.2000000000
From Q4: lambda1 < 30.0000000000
From Q5: lambda1 < 12.3243243243

Maximum lambda1 for ergodicity with lambda2 fixed at 6.0000000000:

lambda1_max = 12.3243243243 clients/sec

limiting queue = Q5

Current lambda1 = 10.0000000000 is below lambda1_max, so the current network is ergodic.

=====
B.6 - End-to-end delay plot versus lambda1
=====

Plot range:

lambda1_min = 1.2324324324 clients/sec
lambda1_max_plot = 12.2010810811 clients/sec
lambda1_max_ergodic = 12.3243243243 clients/sec

Delay values at selected points:

lambda1 = 0.10 * lambda1_max = 1.2324324324 -> E[T] = 0.2950551294 sec
lambda1 = 0.25 * lambda1_max = 3.0810810811 -> E[T] = 0.3323554951 sec
lambda1 = 0.50 * lambda1_max = 6.1621621622 -> E[T] = 0.4349166146 sec
lambda1 = 0.75 * lambda1_max = 9.2432432432 -> E[T] = 0.6880326518 sec
lambda1 = 0.90 * lambda1_max = 11.0918918919 -> E[T] = 1.2973780957 sec
lambda1 = 0.95 * lambda1_max = 11.7081081081 -> E[T] = 2.2074842154 sec
lambda1 = 0.99 * lambda1_max = 12.2010810811 -> E[T] = 9.1519048705 sec

Saved figure as lab5_part2_delay.png

=====
FINAL SUMMARY - Lab 5, Part B
=====

Routing equations:

Gamma1 = lambda1
Gamma2 = lambda2 + (1/2)lambda1
Gamma3 = (5/12)lambda1 + (1/3)lambda2
Gamma4 = (1/3)lambda1 + (2/3)lambda2
Gamma5 = (37/60)lambda1 + (11/15)lambda2

Given parameters:

lambda1 = 10.0000000000, lambda2 = 6.0000000000
mu = [15.0000000000 20.0000000000 10.0000000000 14.0000000000 12.0000000000]

Effective arrival rates:

Gamma = [10.0000000000 11.0000000000 6.1666666667 7.3333333333
10.5666666667]

Traffic intensities:

rho = [0.6666666667 0.5500000000 0.6166666667 0.5238095238 0.8805555556]
ergodic = 1

Mean clients:

E[N_i] = [2.0000000000 1.2222222222 1.6086956522 1.1000000000 7.3720930233]
E[N] = 13.3030108977

End-to-end delay:

E[T] = 0.8314381811 seconds

Bottleneck:
current bottleneck = Q5, $\rho = 0.8805555556$
 $\lambda_{\max} = 12.3243243243$ clients/sec
limiting queue for $\lambda_{\max} = Q5$